.UBAfiuba
FACULTAD DE INGENIERÍA

Sistema modular para monitoreo y control industrial con comunicación inalámbrica y capacidades de automatización embebida

Ing. Lucas Sebastián Kirschner

Carrera de Especialización en Sistemas Embebidos

Director: Mg. Ing. Guillermo Alfredo Fernández (UNaM)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Ciudad de Salto Encantado, octubre de 2026

Resumen

Este documento presenta el desarrollo de un sistema modular para monitoreo y control industrial con comunicación inalámbrica y capacidades de automatización embebida. La solución propuesta permite adquirir datos de campo, controlar actuadores y facilitar la integración de estaciones distribuidas en entornos donde las soluciones cableadas resultan poco prácticas o costosas.

Su desarrollo requirió conocimientos en sistemas embebidos, diseño de hardware electrónico, desarrollo de firmware, sistemas operativos en tiempo real, protocolos de comunicación industrial, redes inalámbricas y diseño de software modular.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen I

1. Introducción general 1
1.1. Automatización industrial . . . 1
1.2. Contexto y motivación . . . 2
1.3. Estado del arte . . . 3
1.3.1. Redes industriales cableadas . . . 3
1.3.2. Redes industriales inalámbricas . . . 3
1.3.3. Protocolos inalámbricos basados en IEEE 802.15.4 . . . 4
1.3.4. Comparación de soluciones existentes . . . 4
1.4. Objetivos del trabajo . . . 5
1.4.1. Objetivo general . . . 5
1.4.2. Objetivos específicos . . . 5

2. Introducción específica 7
2.1. Plataforma de hardware . . . 7
2.2. Componentes y módulos externos . . . 8
2.2.1. SCLT3-8BT8 . . . 8
2.2.2. VNI8200XP-32 . . . 8
2.2.3. SN65HVD82 . . . 8
2.2.4. MRF24J40 . . . 8
2.2.5. LAN8742A . . . 9
2.2.6. Convertidor CC/CC aislado de 5 V . . . 9
2.2.7. Kits de desarrollo y evaluación . . . 9
2.3. Sistema operativo y middleware . . . 9
2.3.1. FreeRTOS . . . 10
2.3.2. CMSIS-RTOS2 . . . 10
2.3.3. HAL . . . 10
2.3.4. lwIP . . . 10
2.4. Protocolos e interfaces de comunicación . . . 11
2.4.1. SPI . . . 11
2.4.2. UART y RS-485 . . . 11
2.4.3. Ethernet y RMII . . . 11
2.4.4. IEEE 802.15.4 . . . 12
2.4.5. GPIO . . . 12
2.4.6. SWD . . . 12

3. Diseño e implementación 13
3.1. Arquitectura del sistema . . . 13
3.1.1. Diagrama de bloques del hardware . . . 13
3.1.2. Diagrama de bloques del firmware . . . 14
3.2. Diseño del hardware . . . 16

3.2.1. Plataforma de procesamiento . . . 16
3.2.2. Sistema de alimentación . . . 17
3.2.3. Entradas digitales . . . 18
3.2.4. Salidas digitales . . . 19
3.2.5. Interfaz RS485 . . . 19
3.2.6. Interfaz inalámbrica . . . 19
3.2.7. Interfaz Ethernet . . . 20
3.2.8. Diseño del PCB . . . 20
3.3. Diseño del firmware . . . 21
3.3.1. Organización general del firmware . . . 22
3.3.2. Organización de las tareas . . . 23
Tarea de entradas digitales . . . 23
Tarea de salidas digitales . . . 24
Tarea de comunicación RS485 . . . 25
Tarea de comunicación inalámbrica . . . 25
Tarea de reloj de tiempo real . . . 26
Tarea de usuario . . . 27
Tarea de diagnóstico . . . 27
3.4. Comunicaciones e interfaces . . . 28
3.4.1. Comunicación inalámbrica . . . 28
3.4.2. Comunicación mediante RS485 . . . 29
3.5. Integración hardware-software . . . 30
3.5.1. Integración de las entradas digitales . . . 30
3.5.2. Integración de las salidas digitales . . . 31
3.5.3. Integración de la interfaz RS485 . . . 31
3.5.4. Integración de la interfaz de radiofrecuencia . . . 32

4. Ensayos y resultados **33**
4.1. Plan de ensayos . . . 33
4.2. Ensayos de hardware . . . 34
4.2.1. Ensayo de las etapas de alimentación y consumo . . . 35
4.2.2. Ensayo de la etapa de entradas digitales . . . 35
4.2.3. Ensayo de la etapa de salidas digitales . . . 35
4.2.4. Ensayo de la interfaz RS485 . . . 36
4.2.5. Ensayo de la interfaz inalámbrica . . . 37
4.3. Ensayos de firmware . . . 37
4.3.1. Pruebas unitarias . . . 38
Controlador de entradas digitales SCLT3-8BT8 . . . 38
Controlador de salidas digitales VNI8200XP-32 . . . 38
Controlador de comunicación RS485 . . . 38
Controlador de comunicación inalámbrica MRF24J40 . . . 39
Resultados de las pruebas unitarias . . . 39
4.3.2. Evaluación temporal del firmware . . . 39
Tiempo de procesamiento de las salidas digitales . . . 40
Tiempo de adquisición de las entradas digitales . . . 40
Latencia de la comunicación inalámbrica . . . 40
4.4. Ensayos de integración . . . 41

Bibliografía **43**

Índice de figuras

1.1. Arquitectura de un sistema distribuido de monitoreo y control industrial. 2

3.1. Arquitectura general del sistema implementado. 13
3.2. Diagrama de bloques general del hardware. 14
3.3. Diagrama de bloques general del firmware. 15
3.4. Asignación de interfaces del microcontrolador a los distintos módulos funcionales. 17
3.5. Esquema eléctrico de la etapa de alimentación. 18
3.6. Vista tridimensional del PCB desarrollado. 21
3.7. Organización general del firmware de la estación central. 22
3.8. Organización general del firmware de las estaciones remotas. . . . 23
3.9. Integración hardware-software de las entradas digitales. 30
3.10. Integración hardware-software de las salidas digitales. 31
3.11. Integración hardware-software de la interfaz RS485. 31
3.12. Integración hardware-software de la interfaz de radiofrecuencia. . 32

4.1. Banco de trabajo e instrumental utilizado para la realización de los ensayos. 34
4.2. Ensayos de las entradas y salidas digitales 36
4.3. Ensayo de la interfaz RS485 36
4.4. Ensayo de la interfaz inalámbrica 37
4.5. Medición de la latencia de comunicación inalámbrica 41

Índice de tablas

1.1. Comparación de tecnologías industriales 5

3.1. Formato de la trama inalámbrica 28
3.2. Unidad de datos RS485 29

4.1. Instrumental y herramientas utilizados en los ensayos 34
4.2. Mediciones de alimentación y consumo 35
4.3. Resultados de las pruebas unitarias de los controladores de firmware. 39

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se presenta el contexto general del trabajo desarrollado y se introducen los conceptos fundamentales relacionados con los sistemas embebidos aplicados a automatización industrial. Se describe la motivación del trabajo, se revisan las principales tecnologías existentes y se establecen los objetivos definidos para el desarrollo del sistema propuesto.

1.1. Automatización industrial

La automatización industrial constituye uno de los pilares fundamentales de los sistemas de producción modernos, ya que permite incrementar la eficiencia, confiabilidad y trazabilidad de los procesos industriales mediante la incorporación de tecnologías de monitoreo, control y comunicación [1]. En el contexto de la Industria 4.0, la integración de dispositivos inteligentes, redes de comunicación y sistemas distribuidos adquirió un rol central en el desarrollo de arquitecturas industriales flexibles y altamente interconectadas [1, 2].

Dentro de este escenario, los sistemas embebidos representan un componente esencial de las plataformas de automatización modernas [3]. Estos sistemas permiten integrar capacidades de adquisición de datos, procesamiento, comunicación y control en dispositivos compactos y especializados, lo que posibilita la implementación de soluciones orientadas a monitoreo de variables de proceso, supervisión remota, control de actuadores y adquisición distribuida de datos.

En paralelo, la evolución de los microcontroladores modernos y de los sistemas operativos de tiempo real permitió desarrollar sistemas embebidos cada vez más complejos y conectados [4]. Actualmente, es posible integrar múltiples interfaces de comunicación y ejecutar tareas concurrentes en tiempo real mediante dispositivos de bajo consumo y reducido tamaño. Asimismo, la utilización de arquitecturas de software modulares y protocolos de comunicación estandarizados facilita la interoperabilidad entre dispositivos y la adaptación de una misma plataforma a diferentes aplicaciones industriales [4, 2].

En este contexto, los sistemas embebidos aplicados a automatización industrial constituyen una herramienta fundamental para el desarrollo de soluciones distribuidas de monitoreo y control. Su utilización permite implementar plataformas flexibles y escalables capaces de integrar procesamiento local, comunicación y supervisión en tiempo real dentro de distintos entornos industriales. Además, las arquitecturas distribuidas y las redes inalámbricas industriales representan una alternativa de creciente interés en aplicaciones asociadas a fábricas inteligentes y sistemas de manufactura avanzados [5].

La figura 1.1 presenta un ejemplo conceptual de una arquitectura distribuida de monitoreo y control industrial, compuesta por una estación central, múltiples estaciones remotas y dispositivos de campo interconectados mediante enlaces de comunicación.



FIGURA 1.1. Arquitectura de un sistema distribuido de monitoreo y control industrial.

1.2. Contexto y motivación

Tradicionalmente, gran parte de los sistemas de automatización industrial se implementaron mediante arquitecturas centralizadas y redes cableadas. Si bien estas soluciones ofrecen altos niveles de robustez y confiabilidad, también presentan limitaciones en entornos donde el tendido de cables resulta costoso, complejo o poco práctico [5]. Esta situación es frecuente en pequeñas y medianas industrias, instalaciones distribuidas geográficamente o procesos que requieren modificaciones frecuentes en la disposición física de los equipos.

En estos escenarios, la incorporación de tecnologías de comunicación inalámbrica permite aumentar la flexibilidad de instalación y reducir significativamente los costos asociados a infraestructura [5]. Además, las arquitecturas distribuidas facilitan la expansión del sistema mediante la incorporación de nuevos nodos sin necesidad de realizar modificaciones importantes en el cableado existente.

Dentro de este contexto, los sistemas distribuidos de monitoreo y control representan una alternativa de creciente interés para aplicaciones industriales. Estas arquitecturas se basan en la utilización de una estación central y múltiples nodos remotos capaces de interactuar con sensores y actuadores ubicados en distintos puntos de una planta o instalación [5]. La información recolectada puede transmitirse hacia sistemas de supervisión o control, lo que permite centralizar el monitoreo del proceso y mejorar la capacidad de diagnóstico y mantenimiento.

La motivación principal del presente trabajo surge de la necesidad de desarrollar una plataforma embebida flexible, escalable y de bajo costo de implementación,

orientada a aplicaciones de monitoreo y control industrial distribuido. El sistema propuesto busca integrar capacidades de adquisición de señales, control de entradas y salidas, procesamiento local y comunicación entre nodos, utilizando una arquitectura modular que facilite su adaptación a diferentes escenarios de aplicación [4]. Particularmente, se busca ofrecer una solución técnicamente viable para pequeñas y medianas industrias, donde los costos asociados a infraestructura y expansión suelen representar una limitación significativa para la incorporación de tecnologías de automatización.

1.3. Estado del arte

La evolución de los sistemas de automatización industrial dio lugar al desarrollo de diversas tecnologías de comunicación destinadas a satisfacer los requerimientos de monitoreo y control de los procesos productivos. En la actualidad conviven soluciones cableadas e inalámbricas, cada una con características particulares en términos de desempeño, flexibilidad y costo de implementación. A continuación, se revisan las tecnologías más representativas relacionadas con la propuesta desarrollada en el presente trabajo.

1.3.1. Redes industriales cableadas

En los entornos de automatización industrial modernos, las redes de comunicación constituyen un elemento fundamental para la adquisición de datos, supervisión y control de procesos. Tradicionalmente, la industria ha adoptado soluciones cableadas debido a su elevada confiabilidad, inmunidad al ruido electromagnético y capacidad para garantizar tiempos de comunicación determinísticos [1, 2].

Las redes industriales cableadas permiten la interconexión de sensores, actuadores, PLC (*Programmable Logic Controller*, controlador lógico programable) y sistemas de supervisión industrial, lo que posibilita el intercambio de información en tiempo real dentro de plantas y procesos automatizados. Entre las tecnologías más utilizadas se destacan buses de campo y soluciones basadas en Ethernet industrial, como PROFIBUS y PROFINET, ampliamente adoptadas en aplicaciones de automatización industrial [6, 7].

La evolución de los sistemas industriales y la aparición de los conceptos de Industria 4.0 e IIoT (*Industrial Internet of Things*, internet industrial de las cosas) impulsaron además la integración de infraestructuras Ethernet, sistemas distribuidos y arquitecturas orientadas a comunicaciones en tiempo real [1]. Sin embargo, a pesar de sus ventajas en términos de robustez y desempeño, las soluciones cableadas presentan limitaciones asociadas a los costos de instalación, mantenimiento y expansión de infraestructura física, especialmente en aplicaciones distribuidas, móviles o de difícil acceso.

1.3.2. Redes industriales inalámbricas

En los últimos años, el avance de los conceptos de Industria 4.0 e IIoT impulsó el crecimiento de las redes inalámbricas industriales como complemento de las infraestructuras cableadas tradicionales [5]. Estas tecnologías permiten reducir costos de instalación, simplificar tareas de mantenimiento y aumentar la flexibilidad de despliegue [8, 9, 10].

Las redes inalámbricas industriales resultan particularmente atractivas para aplicaciones de monitoreo, adquisición distribuida de datos y automatización, debido a ventajas como la movilidad, la facilidad de expansión y la reducción del cableado físico [5]. No obstante, los entornos industriales presentan desafíos significativos asociados a interferencias electromagnéticas, obstáculos físicos, topologías dinámicas y requisitos estrictos de latencia y confiabilidad, lo que impone exigencias adicionales sobre los protocolos de comunicación inalámbrica [5].

Gran parte de las soluciones inalámbricas industriales modernas [11, 12, 13] se basan en el estándar IEEE 802.15.4 [14]. Dicho estándar define las capas física PHY (*Physical Layer*) y de control de acceso al medio MAC (*Medium Access Control*) para redes inalámbricas personales de baja velocidad LR-WPAN (*Low-Rate Wireless Personal Area Network*, red inalámbrica personal de baja velocidad) [14]. Este estándar se caracteriza por su bajo consumo energético, baja complejidad y reducido costo de implementación, características que favorecieron su adopción en sistemas embebidos, redes de sensores inalámbricos e IoT (*Internet of Things*, internet de las cosas) [4]. Asimismo, múltiples protocolos industriales inalámbricos utilizan IEEE 802.15.4 como base tecnológica para la implementación de mecanismos de comunicación orientados a monitoreo y automatización industrial.

1.3.3. Protocolos inalámbricos basados en IEEE 802.15.4

Entre los protocolos inalámbricos industriales más difundidos basados en IEEE 802.15.4 se encuentra WirelessHART [11], ampliamente utilizado en automatización de procesos. Este protocolo incorpora topologías de red mallada (mesh), mecanismos redundantes de comunicación y sincronización temporal, lo que permite aumentar la confiabilidad de la red frente a interferencias o fallas de enlace.

Otra tecnología relevante es ISA100.11a [12, 5], diseñada específicamente para aplicaciones industriales de monitoreo y control inalámbrico, con foco en interoperabilidad, escalabilidad y coexistencia con otras tecnologías de comunicación [5].

Zigbee [13, 5] representa otra de las implementaciones más conocidas sobre IEEE 802.15.4. Aunque su adopción se concentra principalmente en aplicaciones IoT, domótica y monitoreo de bajo consumo, muchas de sus características técnicas, como el bajo costo, el soporte para topologías mesh y el reducido consumo energético, lo convierten en una referencia importante dentro del ecosistema de redes inalámbricas embebidas [5].

1.3.4. Comparación de soluciones existentes

La tabla 1.1 presenta una comparación general entre algunas de las principales tecnologías de comunicación industrial utilizadas en aplicaciones de automatización y monitoreo. Se consideran características relacionadas con el medio de transmisión, las capacidades de tiempo real, el modelo de licenciamiento y el costo de implementación, incluyendo además la propuesta desarrollada en el presente trabajo.

TABLA 1.1. Comparación de tecnologías industriales.

Tecnología	Medio	T. real	Licencia	Costo
PROFIBUS [6]	Cableado	Alto	Propietaria	Alto
PROFINET [7]	Cableado	Muy alto	Propietaria	Alto
WirelessHART [11]	Inalámbrico	Medio	Requerida	Alto
ISA100.11a [12]	Inalámbrico	Medio	Requerida	Alto
Zigbee [13]	Inalámbrico	Bajo	Parcialmente abierta	Medio
Propuesta	Inalámbrico	Medio	Propia	Bajo

La solución propuesta en este trabajo toma como referencia las arquitecturas inalámbricas industriales modernas basadas en IEEE 802.15.4, priorizando una implementación flexible, modular y de bajo costo orientada a aplicaciones embebidas de monitoreo y control distribuido. El desarrollo busca proporcionar una plataforma adaptable a distintos procesos productivos, capaz de integrarse con sensores y actuadores industriales mediante una arquitectura escalable y reutilizable. Asimismo, se plantea como una alternativa de bajo costo de implementación orientada a pequeñas y medianas industrias, donde las limitaciones económicas y de infraestructura suelen dificultar la incorporación de soluciones convencionales de automatización industrial.

1.4. Objetivos del trabajo

Los objetivos definidos para el presente trabajo establecen el propósito general del sistema desarrollado y delimitan las principales actividades necesarias para su diseño, implementación y validación. A continuación, se presentan el objetivo general y los objetivos específicos que guiaron el desarrollo del proyecto.

1.4.1. Objetivo general

Desarrollar un sistema embebido modular y escalable orientado a aplicaciones de monitoreo y control industrial distribuido, basado en una arquitectura compuesta por una estación central y múltiples estaciones remotas interconectadas mediante comunicación inalámbrica [15].

1.4.2. Objetivos específicos

Los objetivos específicos definidos para el desarrollo del presente trabajo son los siguientes [15]:

- Diseñar e implementar un prototipo funcional de estación central basado en un microcontrolador de altas prestaciones capaz de ejecutar un sistema operativo de tiempo real RTOS (*Real-Time Operating System*, sistema operativo de tiempo Real).
- Diseñar e implementar prototipos funcionales de estaciones remotas con capacidad de adquisición de señales y control de entradas y salidas industriales.
- Implementar un sistema de comunicación inalámbrica entre estaciones utilizando tecnología basada en IEEE 802.15.4.

- Desarrollar el firmware necesario para la inicialización y gestión de periféricos, comunicaciones y manejo de entradas y salidas en las estaciones central y remotas.
- Diseñar una arquitectura de software modular que permita abstraer las dependencias de hardware y facilite la reutilización y escalabilidad del sistema.
- Implementar interfaces de conexión para sensores y actuadores industriales que permitan adaptar el sistema a diferentes aplicaciones de monitoreo y control.
- Diseñar e implementar la arquitectura de alimentación de las estaciones.
- Validar el funcionamiento del sistema mediante pruebas experimentales orientadas a verificar la comunicación entre nodos, el procesamiento de señales y el control de entradas y salidas.

Capítulo 2

Introducción específica

En este capítulo se describen las principales plataformas de hardware, componentes externos, capas de software y protocolos de comunicación utilizados durante el desarrollo del trabajo. Asimismo, se presentan las tecnologías de terceros empleadas para la implementación de las funciones de adquisición, procesamiento y comunicación del sistema.

2.1. Plataforma de hardware

Tanto la estación central como las estaciones remotas del sistema utilizan el microcontrolador STM32H563RI de STMicroelectronics, perteneciente a la familia STM32H5 [16, 17, 18]. Esta decisión permitió uniformar la arquitectura de hardware y simplificar el desarrollo y mantenimiento del firmware. Además, facilitó la reutilización de controladores, configuraciones de periféricos y capas de abstracción de hardware entre ambos tipos de nodos.

El STM32H563RI se basa en una arquitectura ARM Cortex-M33 de 32 bits [19], orientada a sistemas embebidos de alto rendimiento y plataformas conectadas. El núcleo incorpora una unidad de punto flotante FPU (*Floating Point Unit*, unidad de punto flotante), extensiones DSP (*Digital Signal Processing*, procesamiento digital de señales) y mecanismos de seguridad basados en TrustZone [18, 20], adecuados para aplicaciones industriales e IoT.

El microcontrolador dispone de memoria Flash integrada y memoria SRAM (*Static Random Access Memory*, memoria estática de acceso aleatorio) interna, además de una amplia variedad de periféricos de comunicación y control [16]. Entre los periféricos utilizados durante el desarrollo se encuentran interfaces SPI (*Serial Peripheral Interface*, interfaz periférica serie), UART (*Universal Asynchronous Receiver-Transmitter*, transmisor-receptor asíncrono universal) y Ethernet MAC (*Media Access Control*, control de acceso al medio).

La selección de este microcontrolador también se fundamentó en la disponibilidad de herramientas de desarrollo y middleware provistos por STMicroelectronics, incluido STM32CubeIDE [21], bibliotecas HAL (*Hardware Abstraction Layer*, capa de abstracción de hardware) [22] y soporte para sistemas operativos de tiempo real como FreeRTOS [23]. Estas herramientas simplificaron significativamente el desarrollo, depuración y mantenimiento del firmware del sistema.

2.2. Componentes y módulos externos

Complementariamente a la plataforma de procesamiento basada en el STM32H-563RI, el sistema incorpora diversos circuitos integrados y módulos externos destinados a la adquisición de señales industriales, el control de entradas y salidas, las comunicaciones cableadas e inalámbricas, la alimentación del sistema y la validación del firmware durante las etapas de desarrollo. A continuación, se describen los componentes más relevantes utilizados en el presente trabajo.

2.2.1. SCLT3-8BT8

El SCLT3-8BT8 de STMicroelectronics es una interfaz de entradas digitales industriales de ocho canales con transferencia serializada de estados mediante interfaz SPI [24]. El dispositivo está orientado a aplicaciones de automatización industrial y PLC. Además, proporciona adaptación y protección para señales de 24 V provenientes de sensores y dispositivos industriales.

El circuito incorpora protección contra sobretensiones y compatibilidad con estándares industriales de inmunidad electromagnética, incluida IEC61000-4-2, IEC-61000-4-4 e IEC61000-4-5. Además, dispone de indicadores de estado integrados y mecanismos de diagnóstico para detección de fallas en las entradas.

2.2.2. VNI8200XP-32

El VNI8200XP-32 de STMicroelectronics es un controlador de salidas digitales industriales de ocho canales basado en transistores MOSFET (*Metal-Oxide-Semiconductor Field-Effect Transistor*, transistor de efecto de campo metal-óxido-semiconductor) de tipo *high-side* [25]. El dispositivo permite accionar cargas industriales de 24 V mediante una interfaz SPI o paralelo configurable.

El componente incorpora funciones de protección y diagnóstico, incluyendo protección térmica, limitación de corriente, detección de sobrecorriente, monitoreo de alimentación y señalización de fallas. También dispone de mecanismos específicos frente a condiciones de cortocircuito y sobretemperatura, características relevantes en aplicaciones industriales.

2.2.3. SN65HVD82

El SN65HVD82 de Texas Instruments es un transceptor RS-485 (*Recommended Standard 485*) diseñado para comunicaciones industriales robustas [26]. El dispositivo opera con alimentación de 5 V y proporciona transmisión diferencial compatible con buses multipunto de larga distancia.

El componente incorpora protección ESD (*Electrostatic Discharge*, descarga electrostática) conforme a normas IEC orientadas a aplicaciones industriales y permite operación en entornos con elevado nivel de ruido electromagnético. Además, soporta altas velocidades de transmisión y presenta bajo consumo energético.

2.2.4. MRF24J40

El MRF24J40 de Microchip es un transceptor inalámbrico compatible con el estándar IEEE 802.15.4 [27]. El dispositivo opera en la banda ISM (*Industrial, Scientific*

and Medical, industrial, científica y médica) de 2.4 GHz y está orientado a aplicaciones de redes inalámbricas de bajo consumo.

El componente incorpora las capas PHY y MAC, además de mecanismos de cifrado AES-128 (*Advanced Encryption Standard*, estándar de cifrado avanzado). La comunicación con el microcontrolador se realiza mediante interfaz SPI.

2.2.5. LAN8742A

El LAN8742A de Microchip es un transceptor Ethernet PHY compatible con Ethernet 10BASE-T y 100BASE-TX [28]. El dispositivo se comunica con el microcontrolador mediante interfaz RMII (*Reduced Media Independent Interface*, interfaz independiente del medio reducida), lo que permite implementar conectividad Ethernet de bajo número de pines.

El componente incorpora soporte para autonegociación, detección automática de polaridad y tecnología HP Auto-MDIX (*Automatic Medium-Dependent Interface Crossover*, cruce automático de interfaz dependiente del medio), característica que permite utilizar cables directos o cruzados.

2.2.6. Convertidor CC/CC aislado de 5 V

El módulo 1769405341 de Würth Elektronik [29] es un convertidor CC/CC aislado empleado para generar la alimentación de 5 V destinada a los circuitos que requieren aislamiento galvánico respecto de la alimentación principal del sistema. El dispositivo incorpora un aislamiento de 1 kV entre entrada y salida y proporciona una potencia de hasta 2 W, características que permiten alimentar las interfaces aisladas manteniendo la separación eléctrica entre ambos dominios de potencia.

2.2.7. Kits de desarrollo y evaluación

Durante las etapas iniciales del proyecto se utilizaron placas STM32 NUCLEO-H563ZI [30], basadas en el microcontrolador STM32H563ZI. Estas placas permitieron realizar pruebas funcionales y tareas de depuración temprana antes del desarrollo del hardware definitivo.

Asimismo, se empleó la placa de expansión X-NUCLEO-PLC01A1 [31], orientada a aplicaciones PLC industriales. Esta placa integra los dispositivos SCLT3-8BT8 [24] y VNI8200XP-32 [25], que proporcionan entradas y salidas industriales de 24 V manejadas mediante SPI. Su utilización permitió validar la arquitectura de firmware y los controladores desarrollados para las interfaces industriales utilizadas posteriormente en el diseño final.

2.3. Sistema operativo y middleware

El desarrollo del firmware del sistema se realizó utilizando distintas capas de software orientadas a facilitar la abstracción del hardware, la administración de tareas concurrentes y la implementación de protocolos de comunicación.

2.3.1. FreeRTOS

FreeRTOS es un sistema operativo de tiempo real ampliamente utilizado en sistemas embebidos [23]. El sistema proporciona mecanismos de planificación multitarea basados en prioridades, sincronización mediante semáforos y mutex, comunicación entre tareas y administración de temporizadores de software.

En el sistema desarrollado, FreeRTOS actuó como núcleo del sistema operativo, mientras que la aplicación utilizó la interfaz estandarizada CMSIS-RTOS2 (*Cortex Microcontroller Software Interface Standard for Real-Time Operating Systems*, estándar de interfaz de software para microcontroladores Cortex para sistemas operativos de tiempo real) para acceder a sus servicios.

2.3.2. CMSIS-RTOS2

El desarrollo de la aplicación en tiempo real se realizó mediante la interfaz CMSIS-RTOS2, especificada por Arm [32]. Esta interfaz define una API estandarizada para sistemas operativos de tiempo real, independiente del núcleo RTOS utilizado.

CMSIS-RTOS2 proporciona mecanismos para la creación y administración de tareas, sincronización mediante semáforos y mutex, comunicación entre tareas, temporizadores y control del tiempo de ejecución. Al definir una interfaz común, permite desacoplar la aplicación del sistema operativo subyacente y facilita la reutilización del código sobre diferentes implementaciones compatibles, como FreeRTOS, CMSIS-RTX y otros sistemas operativos de tiempo real.

En el sistema desarrollado, CMSIS-RTOS2 se utilizó como capa de abstracción entre la aplicación y FreeRTOS. Esta arquitectura permitió desarrollar el firmware utilizando una API estandarizada, reduciendo la dependencia de funciones específicas del RTOS y favoreciendo la portabilidad y el mantenimiento del software.

2.3.3. HAL

Para el acceso y configuración de los periféricos del microcontrolador se utilizaron las bibliotecas HAL provistas por STMicroelectronics [22].

La capa HAL proporciona funciones de alto nivel para el manejo de los periféricos internos del microcontrolador, incluyendo interfaces SPI, UART, Ethernet y temporizadores. Esta capa simplifica el desarrollo del firmware mediante una interfaz uniforme e independiente de los registros específicos del hardware, favoreciendo la portabilidad y el mantenimiento del software.

2.3.4. lwIP

Para la implementación de conectividad Ethernet y protocolos TCP/IP (*Transmission Control Protocol/Internet Protocol*, protocolo de control de transmisión/protocolo de internet) se utilizó el *stack* lwIP (*Lightweight IP*) [33, 34].

lwIP es una implementación liviana del protocolo TCP/IP [33] orientada a sistemas embebidos con recursos limitados. El *stack* incorpora soporte para protocolos IPv4, TCP, UDP (*User Datagram Protocol*, protocolo de datagrama de usuario), ICMP (*Internet Control Message Protocol*, protocolo de mensajes de control de internet), DHCP (*Dynamic Host Configuration Protocol*, protocolo de configuración

dinámica de host) y ARP (*Address Resolution Protocol*, protocolo de resolución de direcciones).

En el sistema desarrollado, lwIP fue integrado junto con FreeRTOS y el periférico Ethernet del STM32H563RI. Esta integración permitió implementar comunicación Ethernet para supervisión, configuración y transmisión de datos mediante redes TCP/IP.

2.4. Protocolos e interfaces de comunicación

El sistema desarrollado utiliza distintas interfaces y protocolos de comunicación orientados a la adquisición de datos, control de periféricos, conectividad Ethernet y depuración del firmware. La selección de cada interfaz se realizó en función de los requerimientos de velocidad, complejidad de implementación, robustez y compatibilidad con los dispositivos externos utilizados en el proyecto.

2.4.1. SPI

La interfaz SPI fue utilizada para la comunicación entre el microcontrolador y distintos circuitos integrados externos, incluyendo las interfaces industriales SCLT3-8BT8 [24] y VNI8200XP-32 [25], así como también el transceptor inalámbrico MRF-24J40 [27].

SPI es un protocolo de comunicación serial síncrono basado en una arquitectura maestro-esclavo, caracterizado por su elevada velocidad de transferencia y baja complejidad de implementación. La interfaz utiliza líneas dedicadas de transmisión, recepción y señal de reloj, además de señales independientes de selección de dispositivo.

2.4.2. UART y RS-485

La interfaz UART fue utilizada junto con el transceptor SN65HVD82 [26] para implementar comunicación RS-485 [35].

UART permite establecer comunicaciones seriales asíncronas mediante transmisión y recepción de datos sin necesidad de señal de reloj compartida. En conjunto con RS-485, posibilita implementar enlaces diferenciales robustos, adecuados para entornos industriales con elevado nivel de ruido electromagnético y largas distancias de cableado.

2.4.3. Ethernet y RMII

La conectividad Ethernet del sistema se implementó mediante el periférico Ethernet MAC integrado en el microcontrolador STM32H563RI [20] y el transceptor Ethernet PHY LAN8742A [28].

La comunicación entre el microcontrolador y el PHY se realizó mediante la interfaz RMII [36], utilizada para transmisión y recepción de datos Ethernet con bajo número de pines.

La capa física Ethernet permite establecer comunicación 10BASE-T y 100BASE-TX sobre medio cableado de par trenzado. Esto facilita la integración con redes Ethernet industriales y servicios basados en TCP/IP.

2.4.4. IEEE 802.15.4

La comunicación inalámbrica entre la estación central y las estaciones remotas se basa en el estándar IEEE 802.15.4 [14]. Este estándar define la capa física PHY y la subcapa de control de acceso al medio MAC para redes inalámbricas personales de baja velocidad. Está orientado a enlaces de corto alcance que requieren bajo consumo energético, reducida complejidad y bajo costo de implementación.

Para el enlace desarrollado se utiliza la banda ISM de 2,4 GHz, en la que el estándar admite una velocidad de transmisión de hasta 250 kbps. La capa PHY establece las características de transmisión y recepción por radiofrecuencia, mientras que la subcapa MAC proporciona funciones de acceso al canal, direccionamiento, validación de tramas y confirmación de recepción. Estas características resultan adecuadas para el intercambio de información de monitoreo y control entre los nodos del sistema.

2.4.5. GPIO

Las interfaces GPIO (*General Purpose Input/Output*, entrada/salida de propósito general) fueron utilizadas para el manejo de señales digitales de control, monitoreo de estados y accionamiento de indicadores luminosos.

Estas señales incluyen líneas de habilitación de periféricos, señales de interrupción, selección de dispositivos SPI, monitoreo de estados de alimentación y control de LEDs indicadores presentes en la placa.

2.4.6. SWD

La programación y depuración del microcontrolador se realizó mediante la interfaz SWD (*Serial Wire Debug*) [37], utilizada por las herramientas de desarrollo STMicroelectronics.

SWD constituye una interfaz de depuración derivada de JTAG (*Joint Test Action Group*) [38]. Esta interfaz reduce la cantidad de señales requeridas y conserva capacidades de programación, ejecución paso a paso, lectura de memoria y monitoreo interno del sistema embebido.

Capítulo 3

Diseño e implementación

En el presente capítulo se describe el diseño e implementación del sistema desarrollado. Se presentan la arquitectura general de hardware y software, los criterios utilizados para la selección de componentes y las principales decisiones de diseño adoptadas durante el desarrollo. Asimismo, se detallan la estructura del firmware, las comunicaciones implementadas y la integración entre los distintos módulos del sistema.

3.1. Arquitectura del sistema

La figura 3.1 presenta la arquitectura general del sistema desarrollado. La solución se compone de una estación central conectada mediante Ethernet TCP/IP a la interfaz de usuario y múltiples estaciones remotas enlazadas de forma inalámbrica mediante IEEE 802.15.4. Cada estación remota interactúa con variables de proceso asociadas a sensores, actuadores e interfaces industriales.

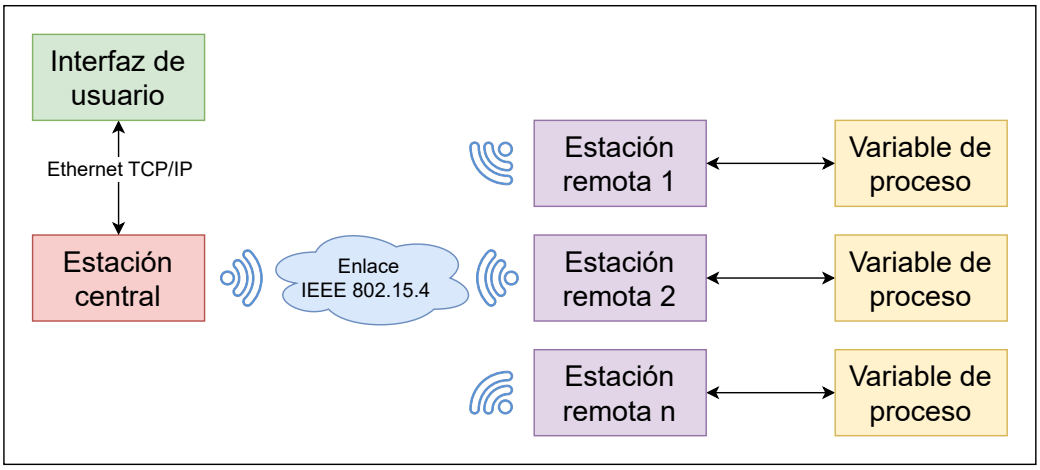


FIGURA 3.1. Arquitectura general del sistema implementado.

3.1.1. Diagrama de bloques del hardware

La figura 3.2 presenta el diagrama de bloques del hardware desarrollado. La arquitectura se basa en un microcontrolador encargado de gestionar las interfaces de comunicación, las entradas y salidas industriales y los módulos auxiliares del sistema. Asimismo, se distinguen los bloques comunes a las estaciones centrales y remotas y aquellos específicos de cada tipo de nodo.

Con el objetivo de uniformar el desarrollo y reducir costos durante una etapa inicial del trabajo, se implementó un único diseño de placa de circuito impreso PCB (*Printed Circuit Board*, placa de circuito impreso) tanto para las estaciones centrales como para las estaciones remotas. Esta estrategia permitió reutilizar el mismo diseño base de hardware y poblar únicamente los componentes requeridos según las funcionalidades de cada estación. Además de simplificar el proceso de desarrollo, esta decisión redujo los costos de fabricación y facilitó las tareas de validación y mantenimiento del sistema.

La estación central incorpora un transceptor Ethernet para la comunicación con una interfaz de usuario ubicada en un nivel jerárquico superior. Asimismo, dispone de un reloj de tiempo real RTC (*Real-Time Clock*, reloj de tiempo real), utilizado para el registro y mantenimiento de fecha y hora ante interrupciones en la alimentación principal.

Por otro lado, las estaciones remotas incorporan una interfaz RS485 para la comunicación con instrumentos industriales y módulos de entradas y salidas digitales destinados a la adquisición y accionamiento de variables de proceso.

Tanto las estaciones centrales como las remotas comparten los módulos de alimentación y regulación de tensión, el microcontrolador principal y el transceptor inalámbrico IEEE 802.15.4 utilizado para la comunicación entre nodos.

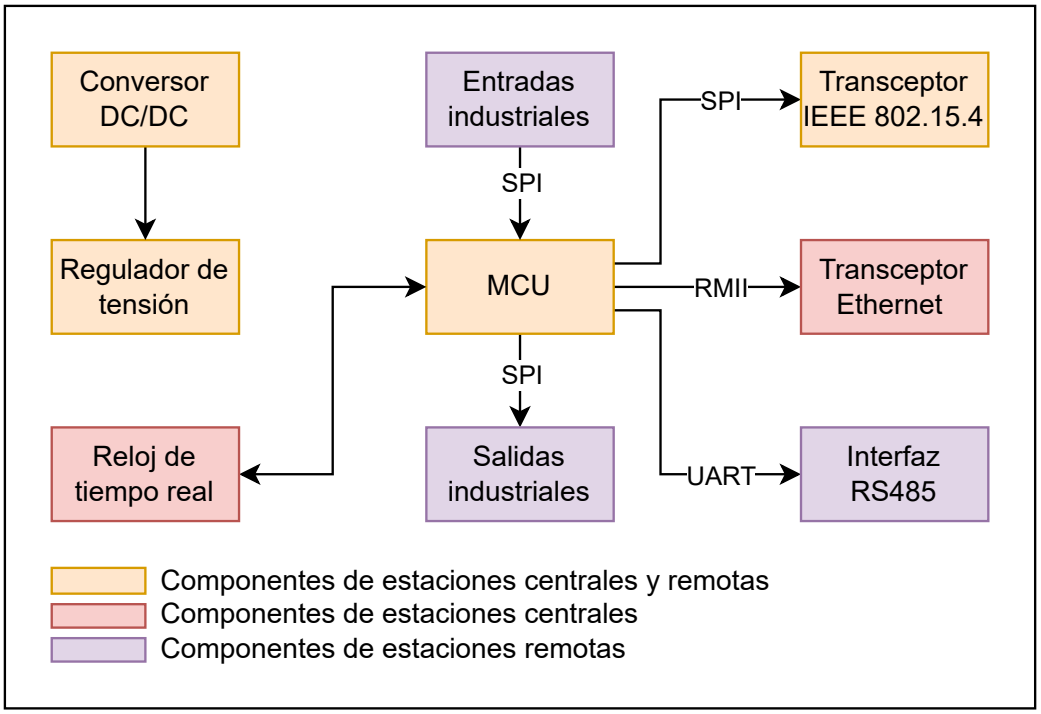


FIGURA 3.2. Diagrama de bloques general del hardware.

3.1.2. Diagrama de bloques del firmware

La figura 3.3 presenta la arquitectura general del firmware desarrollado para el sistema. Este fue organizado en capas con el objetivo de desacoplar la lógica de aplicación de las dependencias específicas del hardware, favorecer la modularidad del software y facilitar la reutilización de código entre estaciones centrales y remotas.

La capa superior implementa la lógica de control y supervisión del sistema. Por debajo se ubica una API (*Application Programming Interface*, interfaz de programación de aplicaciones) encargada de abstraer el acceso a sensores, actuadores e interfaces de comunicación mediante una interfaz uniforme e independiente del hardware subyacente. Esta capa proporciona los servicios necesarios para que la aplicación de usuario pueda implementarse y adaptarse a distintos procesos productivos sin requerir modificaciones en las capas inferiores del firmware.

Las capas intermedias agrupan los servicios de *middleware*, entre los que se encuentran la pila de protocolos lwIP y el sistema operativo en tiempo real FreeRTOS. La interacción entre la aplicación y el sistema operativo se realiza mediante la capa de abstracción CMSIS-RTOS2, que proporciona una interfaz estandarizada e independiente de la implementación particular del RTOS. De esta manera, la aplicación puede desarrollarse sin depender directamente de FreeRTOS, lo que facilita la portabilidad del software hacia otros sistemas operativos compatibles con CMSIS-RTOS2.

Finalmente, las capas inferiores comprenden los controladores de periféricos y comunicaciones, responsables del acceso a las interfaces de hardware, la capa HAL provista por STM32 y, en el nivel más bajo, los módulos físicos que conforman la plataforma, tales como el microcontrolador, el reloj de tiempo real y las interfaces industriales.



FIGURA 3.3. Diagrama de bloques general del firmware.

3.2. Diseño del hardware

El diseño del hardware tuvo como objetivo materializar la arquitectura presentada en la sección anterior mediante una plataforma electrónica robusta, modular y adecuada para aplicaciones industriales. Para ello se seleccionaron componentes con características orientadas al entorno industrial y se adoptaron criterios de diseño que priorizaron la confiabilidad, la inmunidad frente a perturbaciones eléctricas, la facilidad de fabricación y la escalabilidad del sistema.

En las siguientes subsecciones se describen los principales bloques funcionales del hardware desarrollado, abordando las decisiones de diseño adoptadas, la selección de los componentes más relevantes y los criterios empleados para el diseño de la placa de circuito impreso.

3.2.1. Plataforma de procesamiento

El núcleo de procesamiento de la plataforma se implementó mediante el microcontrolador STM32H563ZI de STMicroelectronics, perteneciente a la familia STM32H5. La selección de este dispositivo respondió a la necesidad de disponer de una plataforma con suficiente capacidad de procesamiento para ejecutar simultáneamente tareas de adquisición de datos, control industrial y comunicaciones, permitiendo además incorporar futuras funcionalidades sin requerir modificaciones en la plataforma de procesamiento. La utilización de un mismo microcontrolador tanto en la estación central como en las estaciones remotas permitió unificar la arquitectura de hardware, simplificando el desarrollo del firmware y favoreciendo la reutilización de controladores, configuraciones de periféricos y capas de abstracción entre ambos tipos de nodos.

La disponibilidad de múltiples interfaces de comunicación independientes constituyó uno de los principales criterios para la selección del STM32H563ZI. Esta característica permitió asignar un periférico específico a cada uno de los módulos de la plataforma, evitando compartir recursos críticos y simplificando tanto el diseño del hardware como la implementación del firmware. La figura 3.4 resume esta distribución de periféricos y su interconexión con los principales módulos funcionales del sistema.

Se seleccionó la variante STM32H563ZI en encapsulado LQFP64, cuya disponibilidad de periféricos y líneas de entrada y salida resultó suficiente para implementar todas las funcionalidades previstas en la plataforma. Asimismo, al pertenecer a la misma familia de microcontroladores empleada en la placa de evaluación NUCLEO-H563ZI, fue posible desarrollar y validar las primeras versiones del firmware sobre dicha plataforma y, posteriormente, migrarlas al hardware desarrollado con modificaciones mínimas. Esta estrategia permitió reducir los tiempos de desarrollo, aprovechar los ejemplos y bibliotecas proporcionados por STMicroelectronics y disminuir los riesgos asociados a la puesta en marcha del prototipo.

Todas estas características permitieron disponer de una plataforma de procesamiento capaz de satisfacer los requerimientos funcionales del sistema, manteniendo un elevado grado de reutilización del hardware y del firmware entre las distintas variantes de la plataforma.

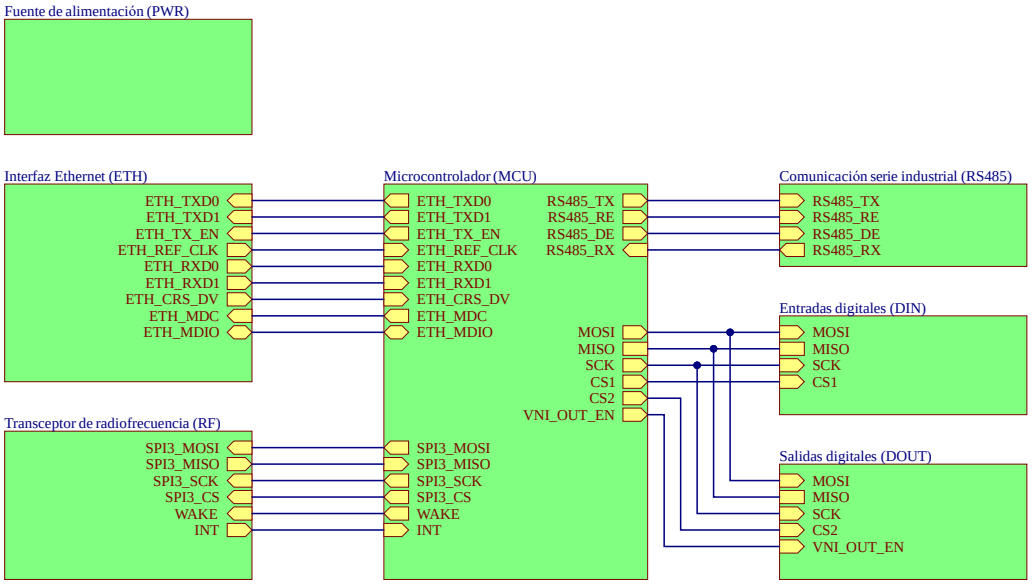


FIGURA 3.4. Asignación de interfaces del microcontrolador a los distintos módulos funcionales.

3.2.2. Sistema de alimentación

Esta etapa fue diseñada para proporcionar un dominio de alimentación aislado de 5 V a partir de una alimentación industrial nominal de 24 V, garantizando además un elevado nivel de robustez frente a las condiciones eléctricas habituales presentes en entornos industriales. La tensión de entrada de 24 V se distribuye directamente hacia las etapas de entradas y salidas industriales, mientras que la electrónica de procesamiento dispone de una alimentación aislada de 5 V, a partir de donde se obtiene posteriormente la tensión de 3,3 V utilizada por el microcontrolador y los circuitos digitales. Esta arquitectura permite desacoplar eléctricamente la lógica de control del dominio de potencia industrial, mejorando la inmunidad frente a perturbaciones y diferencias de potencial. El esquema eléctrico de esta etapa se presenta en la figura 3.5.

La conversión de energía se realiza mediante el módulo convertidor CC-CC aislado 1769405341 de Würth Elektronik, que admite una tensión de entrada nominal de 24 V y entrega una salida regulada de 5 V con una capacidad de corriente de hasta 400 mA. El convertidor incorpora aislamiento galvánico entre la entrada y la salida, proporcionando una tensión de aislamiento de 1 kV, lo que permite desacoplar eléctricamente la electrónica de control respecto del dominio de alimentación industrial y reducir la propagación de perturbaciones y diferencias de potencial provenientes del entorno industrial.

Con el objetivo de aumentar la inmunidad frente a perturbaciones electromagnéticas, tanto la entrada como la salida del convertidor incorporan capacitores de desacople ubicados próximos al módulo. Estos elementos reducen el ruido conducido y estabilizan la tensión de alimentación, lo que contribuye al correcto funcionamiento de la electrónica digital.

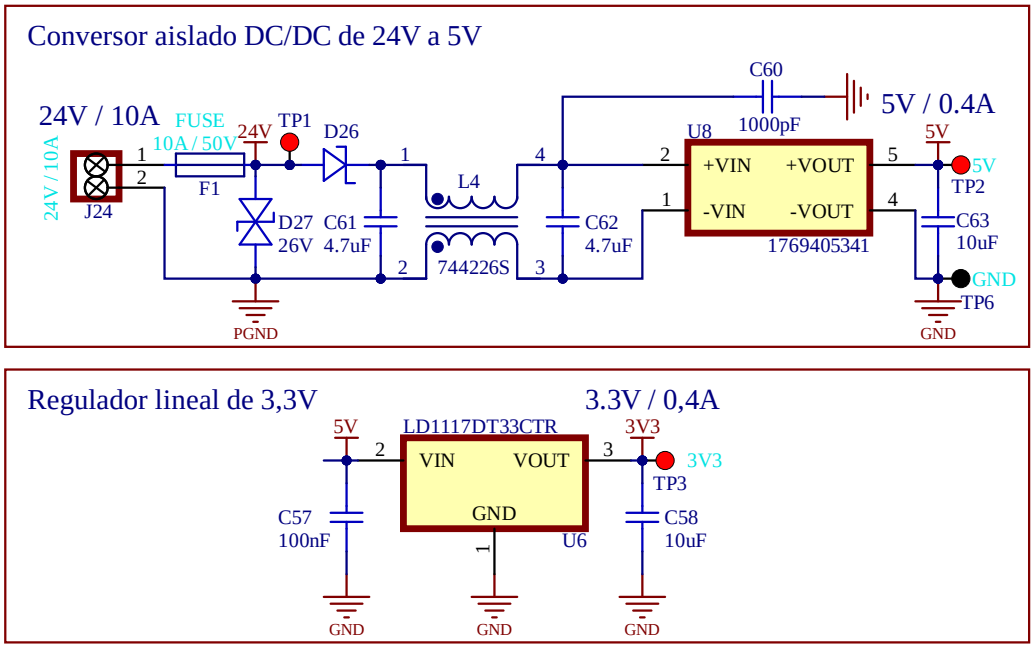


FIGURA 3.5. Esquema eléctrico de la etapa de alimentación.

La protección de la entrada contempla múltiples mecanismos destinados a incrementar la confiabilidad del sistema. Un fusible de 10 A constituye la primera barrera frente a condiciones de sobrecorriente o cortocircuito, limitando la energía disponible durante una falla. A continuación, un diodo TVS de 26 V protege al convertidor frente a sobretensiones transitorias generadas sobre la línea de alimentación, mientras que un diodo *Schottky* conectado en serie impide la circulación de corriente en sentido inverso, proporcionando protección frente a inversiones accidentales de polaridad y evitando daños sobre los circuitos posteriores.

A partir de la tensión aislada de 5 V se obtiene la alimentación de 3,3 V utilizada por el microcontrolador y los circuitos lógicos mediante un regulador lineal LD1117DT33CTR de STMicroelectronics. Dado que la demanda de corriente sobre la línea de 3,3 V resulta relativamente reducida, el empleo de un regulador lineal constituye una solución simple, de bajo costo y con un bajo nivel de ruido, lo que hace innecesario incorporar una segunda etapa de conversión conmutada.

3.2.3. Entradas digitales

Las estaciones remotas fueron concebidas para interactuar directamente con sensores industriales mediante entradas digitales compatibles con señales de 24 V, permitiendo la adquisición confiable de estados discretos provenientes de sensores, interruptores y dispositivos de campo.

Para implementar las entradas digitales se seleccionó el circuito integrado SCLT3-8BT8 de STMicroelectronics, un receptor de ocho canales específicamente desarrollado para aplicaciones de automatización industrial y compatible con los requisitos de la norma IEC 61131-2 [39] para entradas digitales de PLC.

El dispositivo incorpora el acondicionamiento necesario para la conexión directa de señales industriales de 24 V, integrando protección frente a sobretensiones, filtrado de ruido, detección del estado lógico y comunicación con el microcontrolador mediante una interfaz SPI. Esta solución reduce significativamente la

cantidad de componentes discretos necesarios respecto de una implementación convencional basada en optoacopladores, comparadores y circuitos de protección individuales.

3.2.4. Salidas digitales

Las estaciones remotas incorporan salidas digitales destinadas al accionamiento de cargas industriales alimentadas a 24 V, tales como relés, electroválvulas, contactores y otros actuadores utilizados habitualmente en sistemas de automatización.

Las salidas digitales se implementaron mediante el circuito integrado VNI8200XP-32 de STMicroelectronics, un controlador inteligente de ocho salidas *high-side* diseñado para aplicaciones industriales. Cada canal puede operar con tensiones comprendidas entre 10,5 V y 36 V, suministrando hasta 1 A de corriente continua.

El dispositivo integra protección individual frente a sobrecorriente, cortocircuito y sobretemperatura, además de diagnóstico del estado de alimentación y detección de fallas mediante interfaz SPI. Estas características incrementan la confiabilidad del sistema y permiten detectar condiciones anómalas sin necesidad de circuitería adicional.

Debido a que las cargas industriales se alimentan desde una fuente de 24 V independiente de la electrónica de control y la comunicación con el microcontrolador se realiza mediante una interfaz digital SPI aislada del dominio de potencia, la arquitectura implementada preserva el aislamiento galvánico requerido entre la lógica de control y las salidas industriales.

3.2.5. Interfaz RS485

Con el objetivo de permitir la integración de las estaciones remotas con equipos industriales externos, el sistema incorpora una interfaz de comunicación diferencial basada en el estándar RS485.

La comunicación cableada con equipos industriales se implementó mediante un transceptor diferencial SN65HVD82 de Texas Instruments, compatible con el estándar TIA/EIA-485-A [40]. El dispositivo opera sobre buses multipunto de dos hilos y soporta velocidades de transmisión de hasta 200 kb/s en enlaces de aproximadamente 1200 m.

El transceptor se conecta al microcontrolador mediante una interfaz UART y una señal adicional de control de dirección, lo que permite la implementación de enlaces semidúplex compatibles con la mayoría de los instrumentos industriales y dispositivos de automatización disponibles comercialmente.

3.2.6. Interfaz inalámbrica

La comunicación entre la estación central y las estaciones remotas se implementó mediante el módulo de radiofrecuencia MRF24J40MA de Microchip Technology, un transceptor compatible con el estándar IEEE 802.15.4 [14] que opera en la banda ISM de 2,4 GHz. El módulo integra el transceptor de radio, el cristal de referencia, la red de adaptación de impedancias, el regulador interno y una antena impresa sobre la propia placa, lo que reduce la complejidad del diseño de RF y simplifica su integración en el sistema.

El MRF24J40MA se comunica con el microcontrolador mediante una interfaz SPI de cuatro hilos, complementada con señales dedicadas de interrupción, reinicio y activación, permitiendo una integración sencilla con la arquitectura de firmware desarrollada para el proyecto. Asimismo, el módulo soporta una velocidad física de transmisión de 250 kb/s y un alcance de hasta aproximadamente 120 m en condiciones de línea de vista, superando ampliamente el requerimiento de al menos 20 m de cobertura en ambientes interiores establecido para el sistema.

Desde el punto de vista del protocolo de comunicación, la implementación se basa en las capacidades proporcionadas por la capa MAC del estándar IEEE 802.15.4, que incorpora de forma nativa mecanismos de acuse automático de recibo y retransmisión automática de tramas no confirmadas, lo que satisface el requerimiento de confiabilidad definido para el enlace inalámbrico. Adicionalmente, el hardware implementa mecanismos de acceso al medio mediante CSMA/CA, verificación automática de la integridad de las tramas y medición de la calidad del enlace, lo que ayuda a mejorar la robustez de la comunicación en entornos industriales.

3.2.7. Interfaz Ethernet

La estación central incorpora una interfaz Ethernet. Para ello se seleccionó el circuito integrado LAN8742A de Microchip Technology, un transceptor Ethernet (*PHY*) compatible con el estándar IEEE 802.3 [41] para redes 10BASE-T y 100BASE-TX.

El LAN8742A se conecta al microcontrolador mediante la interfaz RMII (*Reduced Media Independent Interface*), mientras que la pila de protocolos TCP/IP es ejecutada por el propio STM32H563 mediante el middleware lwIP. Esta arquitectura permite desacoplar las funciones correspondientes a la capa física de las implementadas por el microcontrolador, proporcionando una solución ampliamente utilizada en sistemas embebidos con conectividad Ethernet.

La interfaz implementada admite velocidades de operación de 10 y 100 Mb/s con negociación automática, cumpliendo el requerimiento de disponer de conectividad Ethernet 10/100 para la integración con sistemas SCADA y otras aplicaciones de supervisión industrial. Asimismo, la utilización de una interfaz cableada dedicada en la estación central permite separar el enlace inalámbrico utilizado para la comunicación con las estaciones remotas de la red de supervisión, lo que simplifica la integración del sistema dentro de infraestructuras industriales existentes.

3.2.8. Diseño del PCB

El diseño del PCB se realizó considerando los requerimientos funcionales del sistema y la necesidad de obtener una solución de bajo costo de fabricación. Para ello, se optó por una placa de dos capas, cuya densidad de interconexión resultó suficiente para integrar todos los módulos de la plataforma sin recurrir a una estructura multicapa.

Durante el diseño se siguieron las recomendaciones establecidas por los estándares IPC-2221A [42] e IPC-2222 [43] para el desarrollo de circuitos impresos, contemplando criterios relacionados con el dimensionamiento de pistas, separación entre conductores, distribución de planos de masa y compatibilidad electromagnética. Asimismo, los *footprints* de los componentes fueron desarrollados

conforme a la norma IPC-7351 [44], mientras que la elaboración de los esquemáticos adoptó la simbología definida por el estándar IEEE Std 315-1975 [45], garantizando uniformidad y compatibilidad con las prácticas habituales de diseño electrónico.

Como resultado de este proceso se obtuvo el diseño de la placa mostrado en la figura 3.6, en el cual se integran los distintos bloques funcionales descritos previamente. La distribución de los componentes se realizó procurando mantener una organización física acorde con las funciones de cada subsistema y facilitar el acceso a los conectores e interfaces externas de la plataforma.

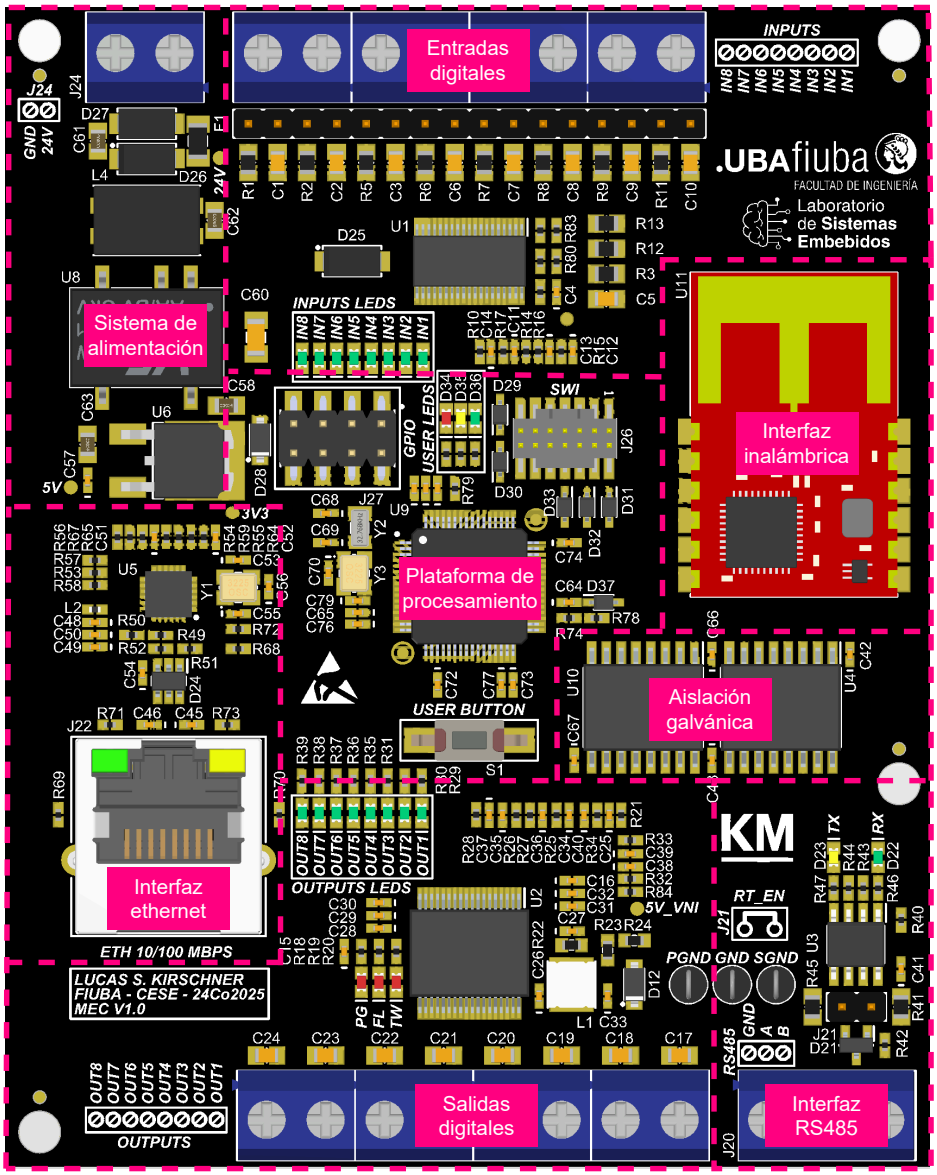


FIGURA 3.6. Vista tridimensional del PCB desarrollado.

3.3. Diseño del firmware

El firmware desarrollado para el sistema fue diseñado con una arquitectura modular orientada a facilitar su mantenimiento, escalabilidad y reutilización entre las distintas variantes de la plataforma. Para ello, se adoptó un sistema operativo

de tiempo real que permite organizar la ejecución del software en múltiples tareas concurrentes, coordinadas mediante mecanismos de sincronización y comunicación entre procesos.

En esta sección se describe la organización general del firmware, el proceso de inicialización del sistema, la distribución de responsabilidades entre las distintas tareas, el manejo de interrupciones, la estructura de los drivers y la lógica de control implementada.

3.3.1. Organización general del firmware

El firmware fue concebido como un conjunto de módulos independientes que ejecutan funciones específicas de manera concurrente sobre FreeRTOS. Esta organización permite distribuir las distintas responsabilidades del sistema entre tareas dedicadas, lo que favorece a una arquitectura modular, escalable y fácilmente mantenible. La interacción entre tareas se realiza exclusivamente mediante interfaces de aplicación bien definidas, evitando el acceso directo a los recursos administrados por otros módulos y promoviendo un bajo acoplamiento entre las distintas funcionalidades del sistema.

Dado que la estación central y las estaciones remotas poseen responsabilidades diferentes, el firmware adopta una organización particular para cada una de ellas. No obstante, ambas implementaciones comparten la misma filosofía de diseño, donde la tarea `userTask` concentra la lógica de aplicación, mientras que el acceso a cada periférico o servicio del sistema se realiza mediante tareas específicas.

La figura 3.7 presenta la organización general del firmware implementado en la estación central. En esta arquitectura, la tarea `userTask` interactúa con la comunicación inalámbrica mediante las funciones `app_rf_send()` y `app_rf_receive()`, delegando su ejecución a la tarea `rfTask`. De forma análoga, las operaciones relacionadas con el reloj en tiempo real se realizan mediante las funciones `app_rtc_set()` y `app_rtc_get()`, administradas por la tarea `rtcTask`. Adicionalmente, la tarea `diagnosticsTask` centraliza el tratamiento de las condiciones de falla notificadas por los distintos módulos mediante los eventos `radioFaultHandle`, `rtcFaultHandle` y `diagnosticsFaultHandle`, lo que proporciona un mecanismo unificado para la supervisión del sistema.

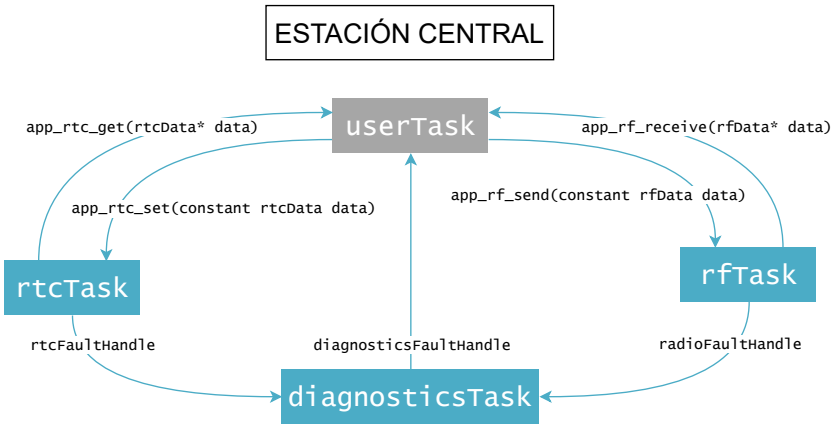


FIGURA 3.7. Organización general del firmware de la estación central.

Por su parte, la figura 3.8 muestra la organización correspondiente a las estaciones remotas. En este caso, además de la tarea `rfTask`, el firmware incorpora las tareas `dinTask`, `doutTask` y `rs485Task`, responsables de administrar las entradas digitales, las salidas digitales y la interfaz RS485, respectivamente. La tarea `userTask` accede a estos recursos mediante las funciones `app_din_read()`, `app_dout_set()`, `app_rs485_send()`, `app_rs485_receive()`, `app_rf_send()` y `app_rf_receive()`, lo que mantiene desacoplada la lógica de aplicación de la implementación específica de cada periférico. Al igual que en la estación central, las distintas tareas reportan las condiciones de error mediante los eventos `dinFaultHandle`, `doutFaultHandle`, `rs485FaultHandle`, `radioFaultHandle` y `diagnosticsFaultHandle`, que son procesadas por la tarea `diagnosticsTask`.

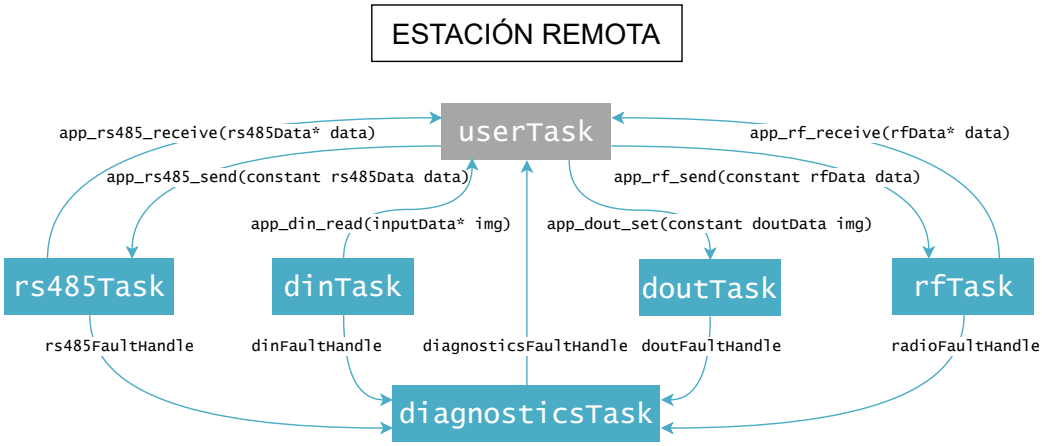


FIGURA 3.8. Organización general del firmware de las estaciones remotas.

En ambas arquitecturas, esta división funcional permite encapsular la gestión de cada subsistema en tareas independientes, simplificando la incorporación de nuevas funcionalidades y facilitando el mantenimiento del firmware. Asimismo, el empleo de interfaces de aplicación estandarizadas reduce las dependencias entre módulos y proporciona una separación clara entre la lógica de control implementada por `userTask` y los mecanismos de acceso a los recursos físicos del sistema.

3.3.2. Organización de las tareas

Las tareas que conforman el firmware presentan distintos modos de ejecución y utilizan los mecanismos proporcionados por CMSIS-RTOS2 para su coordinación y acceso a recursos compartidos. A continuación, se describen las responsabilidades y los principales recursos asociados a cada una de ellas.

Tarea de entradas digitales

La tarea `dinTask` es responsable de mantener actualizado el estado de las entradas digitales de la estación remota. Para ello, realiza periódicamente la lectura del SCLT3-8BT8 y almacena la última imagen de entradas obtenida correctamente. La periodicidad de adquisición se establece mediante un retardo proporcionado por CMSIS-RTOS2, de manera que la tarea permanece bloqueada entre lecturas y libera tiempo de procesamiento para la ejecución del resto de las tareas del sistema.

El acceso a la última imagen válida se encuentra protegido mediante el mutex `dinDataMutex`, ya que esta información puede ser actualizada por `dinTask` y consultada simultáneamente desde otras tareas. La función `app_din_read()` proporciona la interfaz de acceso a estos datos y devuelve una copia de la última imagen disponible sin realizar una nueva adquisición del hardware. De esta manera, las demás tareas pueden consultar el estado de las entradas sin acceder directamente al SCLT3-8BT8 ni interferir con la ejecución periódica de `dinTask`.

Debido a que los circuitos de entradas y salidas digitales comparten el mismo periférico SPI, el acceso físico al bus se encuentra protegido mediante el mutex `ioPortSpiMutex` en la capa encargada de gestionar dicho recurso. Esto evita que las tareas asociadas a ambos subsistemas inicien transferencias simultáneas y garantiza el acceso exclusivo al periférico durante cada operación.

Cuando durante la adquisición se detecta una condición de falla, `dinTask` la informa mediante el grupo de banderas de eventos `dinFault`, lo que hace posible que su tratamiento sea realizado posteriormente por la tarea de diagnóstico. De esta forma, la adquisición de las entradas permanece separada tanto de la lógica de aplicación como del procesamiento de las condiciones de error.

Tarea de salidas digitales

La tarea `doutTask` es responsable de gestionar la actualización de las salidas digitales de la estación remota. A diferencia de la tarea de entradas digitales, su ejecución no es periódica, sino que permanece bloqueada hasta que se recibe una nueva solicitud de actualización. Las demás tareas pueden establecer el estado deseado de las salidas mediante la función `app_dout_set()`, que incorpora la nueva imagen de salidas en la cola de mensajes `outOutputQueue`.

Cuando existe una nueva imagen disponible, `doutTask` se desbloquea, extrae el dato de la cola y realiza la actualización del VNI8200XP-32. Este mecanismo desacopla la generación de una orden de salida de su ejecución sobre el hardware, evitando que la tarea solicitante deba acceder directamente al controlador o esperar a que finalice la transferencia correspondiente. Cuando no existen solicitudes pendientes, la tarea permanece bloqueada y no consume tiempo de procesamiento.

Al igual que en el módulo de entradas digitales, el acceso al periférico SPI compartido se encuentra protegido mediante el mutex `ioPortSpiMutex`. La adquisición y liberación de este recurso se realiza en la capa encargada de gestionar el acceso físico al bus, garantizando que las operaciones sobre el SCLT3-8BT8 y el VNI8200XP-32 se ejecuten de manera mutuamente excluyente. De esta forma, `dinTask` y `doutTask` pueden operar de manera concurrente sin producir transferencias simultáneas sobre el mismo periférico SPI.

Las condiciones de falla detectadas durante la actualización de las salidas, así como los errores asociados al funcionamiento de la cola, se notifican mediante el grupo de banderas de eventos `doutFault`. Esto permite delegar su tratamiento a la tarea de diagnóstico y mantener separadas las responsabilidades de accionamiento y supervisión del sistema.

Tarea de comunicación RS485

La tarea `rs485Task` administra la interfaz RS485 y centraliza el acceso al controlador de comunicación asociado. Su funcionamiento se basa en un esquema asíncrono: la tarea permanece normalmente bloqueada a la espera de eventos y se activa cuando se completa una recepción, existe una solicitud de transmisión, finaliza una transmisión o se detecta una condición de error. Para ello se utiliza el grupo de banderas de eventos `rs485EventHandle`, lo que evita el sondeo continuo del periférico.

El intercambio de datos con las demás tareas se realiza mediante dos colas de mensajes. La función `app_rs485_send()` incorpora los datos pendientes en `rs485OutputQueueHandle` y notifica a `rs485Task` mediante una bandera de evento. Los datos recibidos se almacenan en `rs485InputQueueHandle` y pueden recuperarse mediante la función no bloqueante `app_rs485_receive()`. Esta organización desacopla las tareas de aplicación de las operaciones físicas de transmisión y recepción y evita el acceso directo al driver RS485.

Debido al funcionamiento semidúplex de la interfaz, `rs485Task` también coordina la alternancia entre recepción y transmisión. En condiciones normales se mantiene activa una operación de recepción por interrupciones. Cuando existe información pendiente de transmisión, la recepción se suspende y se inicia el envío del siguiente dato disponible en la cola. Una vez finalizada la transmisión, la tarea procesa las solicitudes restantes o restablece la recepción si no existen nuevos datos pendientes. Las interrupciones asociadas a la UART notifican la finalización de las operaciones mediante las funciones de retorno `rs485_rx_complete_callback()` y `rs485_tx_complete_callback()`. La gestión de las colas y la secuenciación de las operaciones se realizan desde el contexto de `rs485Task`.

Las fallas detectadas durante la inicialización o las operaciones de comunicación, junto con los errores asociados a las colas y los servicios del sistema operativo, se informan mediante el grupo de banderas `rs485FaultHandle`. Su tratamiento puede delegarse a la tarea de diagnóstico, lo que mantiene separadas las responsabilidades de comunicación, lógica de aplicación y supervisión del sistema.

Tarea de comunicación inalámbrica

La tarea `radioTask` administra la interfaz de comunicación inalámbrica y centraliza el acceso al transceptor MRF24J40. Su funcionamiento se basa en un esquema orientado a eventos, por lo que permanece normalmente bloqueada hasta que se produce una interrupción del transceptor o existe información pendiente de transmisión. Para ello se utiliza el grupo de banderas de eventos `radioEventHandle`, lo que evita el sondeo continuo del dispositivo y libera tiempo de procesamiento mientras no existen operaciones pendientes.

El intercambio de información con las demás tareas se realiza mediante dos colas de mensajes. La función `app_radio_send()` incorpora en `radioOutputQueueHandle` el dato que se desea transmitir junto con la dirección del nodo de destino y notifica a `radioTask` mediante una bandera de evento. Los datos recibidos correctamente se almacenan en `radioInputQueueHandle` junto con la dirección del nodo de origen y pueden recuperarse mediante la función no

bloqueante `app_radio_receive()`. Esta organización permite intercambiar información a través del enlace inalámbrico sin acceder directamente al controlador MRF24J40.

La recepción de datos y la finalización de las transmisiones son notificadas por el transceptor mediante su línea de interrupción. La interrupción externa únicamente registra la condición pendiente y establece la bandera correspondiente en `radioEventHandle`; el procesamiento se realiza posteriormente desde el contexto de `radioTask`. La tarea determina entonces el origen de la interrupción y, según corresponda, procesa la información recibida o finaliza la transmisión en curso. Este mecanismo reduce el procesamiento realizado en contexto de interrupción y concentra las operaciones de acceso al transceptor en una única tarea.

Las solicitudes de transmisión se procesan secuencialmente y se mantiene una única operación activa sobre el transceptor. Una vez finalizada cada transmisión, `radioTask` puede iniciar el envío del siguiente elemento almacenado en `radioOutputQueueHandle`. La cola desacopla así la generación de solicitudes por parte de las tareas de aplicación de su ejecución sobre el hardware, mientras que `radioTask` mantiene el control exclusivo del MRF24J40.

Las fallas detectadas durante el acceso al transceptor, el procesamiento de los datos o la utilización de los recursos del sistema operativo se informan mediante el grupo de banderas `radioFaultHandle`. Este recurso también permite registrar condiciones asociadas al resultado de las transmisiones. Su tratamiento puede delegarse a la tarea de diagnóstico, lo que mantiene separadas las responsabilidades de comunicación inalámbrica, lógica de aplicación y supervisión del sistema.

Tarea de reloj de tiempo real

La tarea `rtcTask` mantiene actualizada la información de fecha y hora utilizada por la estación central. Para ello emplea el periférico RTC integrado en el microcontrolador, cuya base de tiempo se obtiene mediante un cristal externo de 32,768 kHz. El dominio correspondiente dispone de alimentación de respaldo por batería, lo que permite conservar la referencia temporal ante la interrupción de la alimentación principal del sistema.

La tarea realiza periódicamente la lectura del RTC y mantiene una copia actualizada de la información temporal para el resto del firmware. Las demás tareas pueden consultarla mediante la función `app_rtc_get()`, sin acceder directamente al periférico. A su vez, la función `app_rtc_set()` permite establecer una nueva fecha y hora cuando sea necesario. De esta manera, las operaciones relacionadas con la referencia temporal quedan centralizadas en este módulo.

El acceso a la información temporal se gestiona para evitar inconsistencias entre su actualización por `rtcTask` y su consulta desde otras tareas. Así, el resto del firmware dispone de una interfaz común para obtener o modificar la fecha y hora, mientras que los detalles del manejo del RTC permanecen encapsulados en el módulo correspondiente.

Las fallas detectadas durante la configuración o el acceso al RTC se informan mediante el grupo de banderas `rtcFaultHandle`. Su tratamiento puede delegarse a la tarea de diagnóstico, lo que mantiene separadas las responsabilidades asociadas a la referencia temporal y a la supervisión general del sistema.

Tarea de usuario

La tarea `userTask` ejecuta la lógica de aplicación específica del proceso productivo en el que se utiliza el sistema. A diferencia de las tareas anteriores, no se encuentra asociada directamente a un periférico o subsistema particular, sino que utiliza las interfaces proporcionadas por los distintos módulos del firmware para implementar las funciones de adquisición, control y comunicación requeridas por cada aplicación.

El acceso a los recursos del sistema se realiza exclusivamente mediante las interfaces de aplicación correspondientes. `userTask` puede consultar el estado de las entradas digitales mediante `app_din_read()`, establecer las salidas mediante `app_dout_set()`, intercambiar información mediante `app_radio_send()`, `app_radio_receive()`, `app_rs485_send()` y `app_rs485_receive()`, y acceder a la referencia temporal mediante `app_rtc_get()` y `app_rtc_set()`. De esta forma, la lógica de usuario permanece desacoplada del acceso directo a los periféricos y de los detalles internos de cada controlador.

La estructura de `userTask` contempla una etapa de inicialización seguida de un ciclo de ejecución periódico, en el que pueden incorporarse las funciones requeridas por el proceso. El retardo entre ejecuciones se implementa mediante los servicios proporcionados por CMSIS-RTOS2, lo que permite definir una periodicidad acorde con las necesidades de la aplicación y evita la ocupación continua del procesador.

Esta organización permite adaptar el firmware a diferentes procesos productivos mediante la modificación principal del comportamiento de `userTask`, mientras que las tareas encargadas de administrar los recursos de hardware pueden mantenerse sin cambios. La tarea de usuario constituye así el nivel de integración entre los servicios proporcionados por la plataforma y la lógica de control específica de cada aplicación.

Tarea de diagnóstico

La tarea `diagnosticsTask` centraliza la supervisión de las condiciones de falla detectadas por los distintos módulos del firmware. Para ello consulta los grupos de banderas asociados a las entradas digitales, las salidas digitales, la comunicación inalámbrica, la interfaz RS485 y el reloj de tiempo real mediante los objetos `dinFaultHandle`, `doutFaultHandle`, `radioFaultHandle`, `rs485FaultHandle` y `rtcFaultHandle`, respectivamente.

En la implementación actual, `diagnosticsTask` captura las banderas de error pendientes y las reporta mediante la interfaz de depuración SWO. Este mecanismo resulta útil durante las etapas de desarrollo y validación, ya que permite identificar el origen y el tipo de falla sin incorporar procesamiento adicional en las tareas que administran cada subsistema.

Como evolución de esta implementación, se prevé incorporar una estrategia de gestión de fallas que permita clasificarlas según su severidad, registrar su estado y notificar las condiciones relevantes a `userTask`. De esta forma, la lógica de aplicación podrá adoptar las acciones correspondientes ante situaciones anómalas, mientras que la detección y consolidación de los errores permanecerán centralizadas en una única tarea.

3.4. Comunicaciones e interfaces

La comunicación entre los distintos elementos del sistema se implementó mediante dos interfaces con objetivos diferentes. El enlace inalámbrico entre la estación central y las estaciones remotas utiliza una implementación basada en IEEE 802.15.4 sobre el transceptor MRF24J40, con direccionamiento de nodos y mecanismos de confirmación de las transmisiones. Por su parte, las estaciones remotas disponen de una interfaz RS485 destinada a la comunicación cableada con dispositivos externos, sobre la cual se implementó inicialmente un mecanismo propietario básico de intercambio de datos.

En ambos casos, las capas de comunicación se diseñaron para desacoplar la lógica de aplicación de los detalles asociados al acceso físico a los periféricos. Las operaciones de transmisión y recepción se gestionan de forma asíncrona y los errores detectados se propagan hacia los mecanismos de diagnóstico descritos previamente.

3.4.1. Comunicación inalámbrica

La comunicación inalámbrica se implementó mediante una red con topología en estrella, compuesta por una estación central configurada como coordinador PAN y hasta dieciséis estaciones remotas. Todos los nodos pertenecen a una misma PAN, identificada mediante un valor fijo, y utilizan direccionamiento corto de 16 bits. La estación central utiliza la dirección `0x0000`, mientras que las estaciones remotas emplean direcciones comprendidas entre `0x0001` y `0x0010`. El rol y la dirección local de cada nodo se establecen durante la compilación del firmware.

La red opera en modo *nonbeacon*, sin mecanismos de asociación, descubrimiento ni asignación dinámica de direcciones. En consecuencia, la estación central conoce previamente las direcciones de los nodos de la red y cada estación remota se comunica exclusivamente con el coordinador. Este esquema simplifica la implementación y resulta suficiente para la topología fija definida para el sistema.

Sobre la capa MAC de IEEE 802.15.4 se implementó una trama de aplicación mínima destinada al intercambio de un byte de información. Su estructura se presenta en la tabla 3.1. La cabecera MAC ocupa nueve bytes e incorpora el campo de control de trama, el número de secuencia, el identificador de la PAN y las direcciones cortas de destino y origen. A continuación, se incorpora un byte correspondiente a la información de aplicación. El campo FCS (*Frame Check Sequence*, secuencia de verificación de trama) no es incorporado por el firmware, ya que el MRF24J40 lo genera y verifica automáticamente.

TABLA 3.1. Formato de la trama utilizada para la comunicación inalámbrica.

Campo	Tamaño	Descripción
Frame Control	2 bytes	Configuración de la trama IEEE 802.15.4
Sequence Number	1 byte	Número de secuencia
Destination PAN ID	2 bytes	Identificador de la red
Destination Address	2 bytes	Dirección corta del nodo de destino
Source Address	2 bytes	Dirección corta del nodo de origen
Payload	1 byte	Dato de aplicación

La transmisión se inicia mediante `app_radio_send()`, que recibe la dirección de destino y el byte que se desea enviar. La información se almacena en una cola hasta que `radioTask` puede procesarla. La tarea construye la trama correspondiente y la carga en la FIFO de transmisión del MRF24J40. Durante la recepción, el transceptor genera una interrupción cuando existe una nueva trama disponible. Posteriormente, `radioTask` recupera la información, verifica que la trama corresponda al formato esperado y extrae la dirección de origen y el dato recibido. Esta información queda disponible para la aplicación mediante `app_radio_receive()`.

Todas las tramas de datos se transmiten con solicitud de acuse de recibo. La generación y detección del ACK, así como las retransmisiones ante la ausencia de confirmación, son realizadas automáticamente por el MRF24J40. Una vez finalizado este procedimiento, el firmware consulta el resultado y distingue entre una transmisión exitosa, una falla por ausencia de ACK luego de agotar las retransmisiones y una falla de acceso al medio asociada al mecanismo CSMA/CA. Estas condiciones, junto con los errores de acceso al hardware, las tramas inválidas y los desbordamientos de las colas, se informan mediante `radioFaultHandle` para su posterior tratamiento por el sistema de diagnóstico.

3.4.2. Comunicación mediante RS485

La interfaz RS485 se implementó inicialmente mediante un mecanismo propietario básico orientado al intercambio asíncrono de datos entre la estación remota y dispositivos externos. En su estado actual, cada operación de transmisión o recepción procesa un único byte, por lo que todavía no se incorporaron campos adicionales de direccionamiento, delimitación de trama, comandos ni mecanismos de verificación mediante suma de comprobación o CRC. Esta solución constituye una interfaz mínima sobre la que pueden incorporarse posteriormente protocolos de mayor complejidad sin modificar las capas encargadas del acceso al hardware.

La transmisión y la recepción utilizan la UART del microcontrolador mediante interrupciones. La capa de bajo nivel controla además las señales DE y /RE del transceptor RS485 para alternar entre los modos de transmisión y recepción. Debido al funcionamiento semidúplex de la interfaz, solo puede existir una operación activa en cada instante. El estado de reposo corresponde al modo de recepción, con el transmisor deshabilitado y el receptor habilitado.

El formato utilizado actualmente se limita al byte de aplicación indicado en la tabla 3.2. Su interpretación depende de la lógica implementada en las capas superiores y no forma parte del driver RS485.

TABLA 3.2. Unidad de datos utilizada por la interfaz RS485.

Campo	Tamaño	Descripción
Payload	1 byte	Dato intercambiado con el dispositivo externo

La función `app_rs485_send()` incorpora el dato en la cola de transmisión y retorna inmediatamente, sin esperar a que finalice la transferencia física. Cuando `rs485Task` procesa la solicitud, suspende una eventual recepción activa, configura el transceptor en modo de transmisión e inicia el envío mediante interrupciones. Una vez transmitido el byte, el controlador retorna al modo de recepción

y notifica a la tarea para que procese el siguiente dato pendiente o restablezca la espera de una nueva recepción.

La recepción permanece normalmente activa mientras no existan transmisiones pendientes. Cuando la UART recibe un byte, genera una interrupción y la información se procesa posteriormente desde el contexto de `rs485Task`. El dato se almacena en la cola de recepción y queda disponible para las demás tareas mediante `app_rs485_receive()`.

El manejo de errores contempla condiciones asociadas a la inicialización, estados inválidos del controlador, tiempos de espera, fallas de hardware y errores informados por la UART. Ante una condición anómala, la operación activa se considera finalizada, el transceptor retorna al modo de recepción y la condición correspondiente se informa mediante `rs485FaultHandle`. También se supervisan los posibles desbordamientos de las colas de transmisión y recepción y los errores asociados a los servicios del sistema operativo.

3.5. Integración hardware-software

La integración entre los módulos de hardware y software se realizó de acuerdo con la arquitectura en capas presentada en las secciones anteriores. Los periféricos físicos son gestionados mediante controladores que encapsulan las operaciones necesarias para su acceso. Sobre estos se implementaron los servicios de aplicación encargados de administrar los principales recursos del sistema, como las entradas y salidas digitales, las interfaces de comunicación y el reloj de tiempo real.

Las distintas funciones se ejecutan de forma concurrente mediante tareas de FreeRTOS, comunicadas principalmente a través de colas de mensajes y grupos de eventos. Esta organización permite desacoplar las solicitudes de su procesamiento y mantener el acceso al hardware dentro de las tareas correspondientes. Las interrupciones de los periféricos se utilizan para señalizar los eventos que requieren atención. En el nivel superior, `userTask` utiliza las interfaces proporcionadas por estos servicios para implementar la lógica específica del proceso productivo.

3.5.1. Integración de las entradas digitales

La integración de las entradas digitales se representa en la figura 3.9. El flujo de información comienza en el circuito SCLT3-8BT8, encargado de acondicionar las señales provenientes del proceso y proporcionar su estado al microcontrolador mediante SPI. Las operaciones sobre esta interfaz se realizan a través de la HAL de STM32 y son encapsuladas por el driver de entradas, lo que evita que las capas superiores dependan de las particularidades de acceso al circuito integrado.

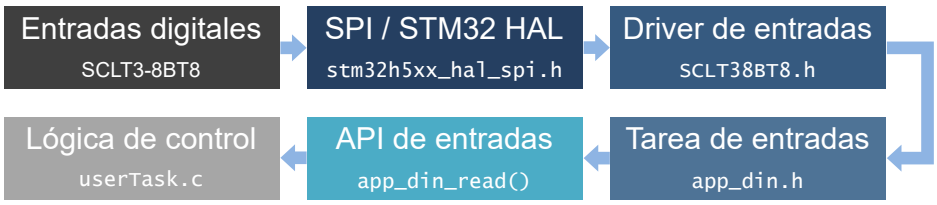


FIGURA 3.9. Integración hardware-software de las entradas digitales.

Sobre este driver se encuentra la tarea de entradas, encargada de realizar la adquisición y mantener disponible el estado de los canales. La API proporciona la función `app_din_read()`, mediante la cual la lógica implementada en `userTask` puede consultar las entradas del sistema. De esta forma, la información fluye desde las señales físicas hacia la lógica de control sin que esta última deba conocer el mecanismo de adquisición, la interfaz SPI ni las características particulares del SCLT3-8BT8.

3.5.2. Integración de las salidas digitales

El flujo asociado a las salidas digitales, presentado en la figura 3.10, se desarrolla en sentido inverso al correspondiente a las entradas. Una decisión generada por la lógica de control en `userTask` se comunica mediante la función `app_dout_set()` de la API de salidas, que permite establecer el estado deseado sin acceder directamente al dispositivo físico.

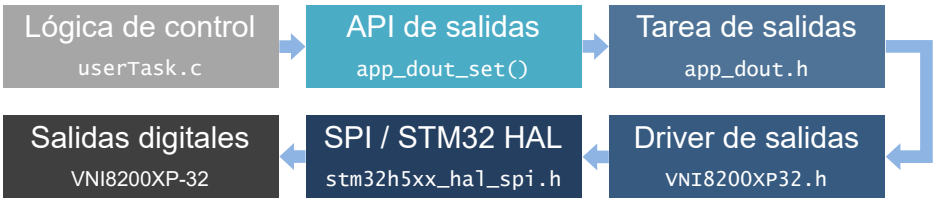


FIGURA 3.10. Integración hardware-software de las salidas digitales.

La solicitud es procesada por la tarea de salidas y transferida al driver del VNI8200XP-32, que utiliza la interfaz SPI y los servicios de la HAL para efectuar la escritura sobre el circuito integrado. Este realiza finalmente el accionamiento de las cargas conectadas a las salidas digitales. Esta separación permite que la lógica del proceso opere sobre estados lógicos de salida, mientras que la transferencia de datos y el control del dispositivo permanecen encapsulados en las capas inferiores.

3.5.3. Integración de la interfaz RS485

A diferencia de los casos anteriores, la interfaz RS485 presenta un flujo de información bidireccional, como se muestra en la figura 3.11. El transceptor SN65HVD82 constituye la interfaz física con el bus y se encuentra conectado a una UART del microcontrolador. La HAL proporciona los mecanismos de transmisión y recepción, mientras que el driver RS485 administra el intercambio de datos y la alternancia entre los modos de transmisión y recepción del transceptor.



FIGURA 3.11. Integración hardware-software de la interfaz RS485.

La tarea de RS485 actúa como intermediaria entre el driver y la lógica de aplicación y gestiona los datos recibidos y las solicitudes pendientes de transmisión.

Las funciones `app_rs485_send()` y `app_rs485_receive()` proporcionan a `userTask` una interfaz bidireccional independiente de los detalles relacionados con la UART, las interrupciones y las señales de habilitación del transceptor. De este modo, la lógica del proceso puede intercambiar información con dispositivos externos mediante un servicio de comunicación uniforme.

3.5.4. Integración de la interfaz de radiofrecuencia

La integración de la comunicación inalámbrica, presentada en la figura 3.12, emplea el transceptor MRF24J40MA-I/RM conectado al microcontrolador mediante SPI. Las operaciones de acceso al dispositivo se realizan a través de la HAL y son encapsuladas por el driver MRF24J40, que proporciona las primitivas necesarias para la transmisión, recepción y gestión de los eventos generados por el transceptor.



FIGURA 3.12. Integración hardware-software de la interfaz de radiofrecuencia.

La tarea de radiofrecuencia concentra el procesamiento asociado al enlace inalámbrico y proporciona a las capas superiores una interfaz mediante las funciones `app_radio_send()` y `app_radio_receive()`. A través de estas funciones, `userTask` puede intercambiar información con otros nodos sin intervenir directamente en el acceso SPI, la atención de interrupciones ni la gestión interna del transceptor. Así, la comunicación inalámbrica queda integrada como un servicio disponible para la lógica específica del proceso productivo.

Capítulo 4

Ensayos y resultados

En este capítulo se presentan los ensayos realizados para verificar el funcionamiento del hardware y del firmware desarrollados, así como su integración en el sistema completo. Las pruebas comprenden la evaluación de los principales bloques de hardware, las funciones implementadas en el firmware y la comunicación entre la estación central y las estaciones remotas. Finalmente, se analizan los resultados obtenidos y los principales desvíos y limitaciones identificados durante la validación.

4.1. Plan de ensayos

La validación del sistema siguió una metodología incremental. Primero se verificaron individualmente los bloques de hardware y firmware, luego se realizaron ensayos de integración entre ambos niveles y, finalmente, se evaluó el sistema completo para comprobar el intercambio de información y la ejecución coordinada de las funciones entre la estación central y las estaciones remotas.

Para cada ensayo se establecieron las condiciones iniciales, la configuración, el procedimiento de prueba y los resultados esperados. Las pruebas de hardware se orientaron a verificar las etapas de alimentación, las entradas y salidas digitales y las interfaces de comunicación. Los ensayos de firmware contemplaron la comprobación de los controladores y servicios desarrollados, la ejecución de las tareas de FreeRTOS y la temporización de las operaciones relevantes. Por último, las pruebas de integración evaluaron el funcionamiento conjunto de estos elementos mediante operaciones representativas del sistema.

Los ensayos se realizaron en condiciones controladas de laboratorio con el instrumental y las herramientas indicados en la tabla 4.1. La figura 4.1 muestra el banco de trabajo utilizado. Los criterios de aceptación se establecieron a partir de los requerimientos funcionales del sistema y las especificaciones de los componentes. Según el ensayo, se consideraron los niveles de tensión y corriente, las características temporales de las señales, la ejecución de las funciones y el intercambio de información entre los módulos.

Debido a la falta de disponibilidad de los circuitos integrados SCLT3-8BT8 y VNI8200XP-32 durante la fabricación de los prototipos, las placas fueron entregadas sin estos componentes. Por este motivo, los ensayos de entradas y salidas digitales se realizaron sobre la plataforma de evaluación NUCLEO-PLC01, que incorpora ambos dispositivos. Esta alternativa permitió verificar su funcionamiento y la comunicación mediante SPI, aunque la validación de estas etapas sobre el PCB desarrollado quedó pendiente.

TABLA 4.1. Instrumental y herramientas utilizados para la realización de los ensayos.

Instrumento o herramienta	Utilización
Fuente regulada de laboratorio	Alimentación del sistema
Multímetro digital	Medición de tensión y corriente
Osciloscopio digital	Análisis temporal de señales
Analizador lógico USB de 8 canales	Análisis de señales digitales
Generador de señales digitales	Estimulación de entradas
STLINK-V3MINIE	Programación y depuración
STM32CubeIDE	Depuración del firmware
Vistas de FreeRTOS	Supervisión de objetos del RTOS
Interfaz SWO	Registro de eventos y variables
Contador de ciclos DWT	Medición de tiempos de ejecución

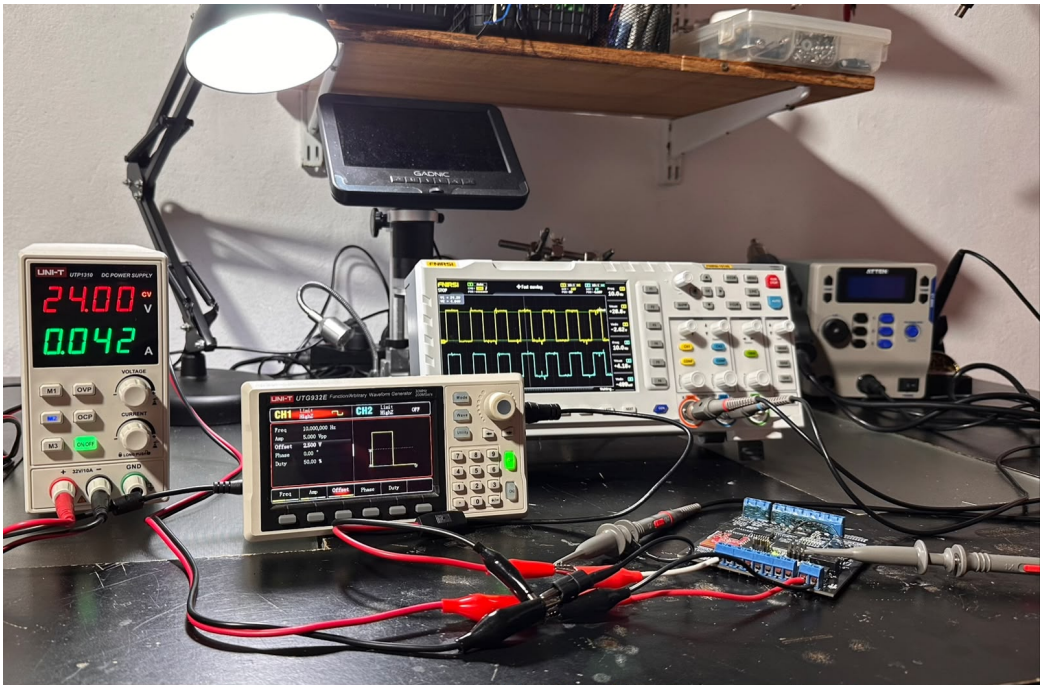


FIGURA 4.1. Banco de trabajo e instrumental utilizado para la realización de los ensayos.

4.2. Ensayos de hardware

En esta sección se presentan los ensayos realizados sobre los principales bloques de hardware del sistema. Las pruebas comprenden la verificación de las tensiones de alimentación y el consumo de corriente, el funcionamiento de las entradas y salidas digitales y las señales asociadas a las interfaces de comunicación. Para cada ensayo se indican las condiciones de medición y los resultados obtenidos mediante el instrumental presentado en la sección anterior.

4.2.1. Ensayo de las etapas de alimentación y consumo

Como primera etapa de validación del hardware, se verificaron las principales tensiones de alimentación y el consumo de corriente de la placa. El sistema se alimentó con 24 V y se midieron, mediante un multímetro digital, las líneas de 5 V y 3,3 V, junto con la corriente consumida desde la alimentación principal. Esta última medición se realizó con la placa en estado de reposo, sin cargas externas conectadas ni operaciones activas de adquisición, actuación o comunicación. Los valores obtenidos se presentan en la tabla 4.2.

TABLA 4.2. Mediciones de las tensiones de alimentación y del consumo de corriente de la placa.

Magnitud	Nominal	Medido
Alimentación de entrada	24 V	24 V
Alimentación de 5 V	5 V	5,02 V
Alimentación de 3,3 V	3,3 V	3,31 V
Consumo a 24 V	–	21 mA

Los resultados obtenidos permitieron verificar el correcto funcionamiento de las etapas de alimentación en las condiciones consideradas. Las líneas de 5 V y 3,3 V presentaron valores próximos a sus niveles nominales, mientras que el consumo base medido sobre la alimentación de entrada de 24 V fue de 21 mA.

4.2.2. Ensayo de la etapa de entradas digitales

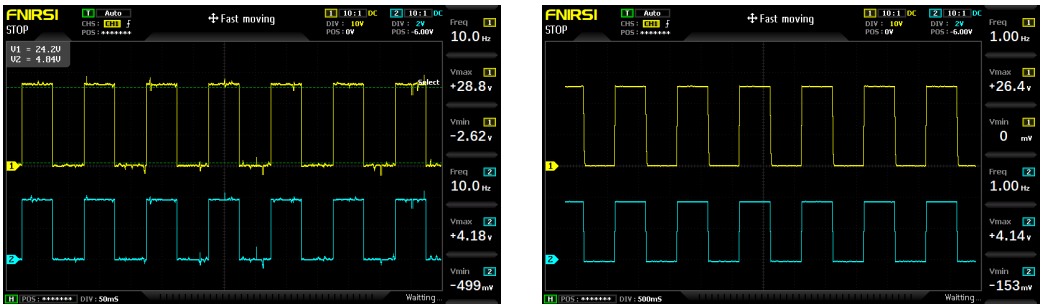
Para verificar el funcionamiento de la etapa de entradas digitales, se aplicó al canal 1 una señal cuadrada de 10 Hz con niveles de 0 V y 24 V. El microcontrolador realizó periódicamente la lectura del SCLT3-8BT8 mediante la interfaz SPI, dentro de un *superloop*, y replicó el estado detectado sobre el pin GPIO0 de la placa.

La figura 4.2a muestra las señales registradas con el osciloscopio. El canal 1 (amarillo) corresponde a la señal aplicada a la entrada digital, mientras que el canal 2 (celeste) representa el estado obtenido en GPIO0. La correspondencia entre ambas formas de onda permitió comprobar la detección de los niveles de 0 V y 24 V, su adquisición mediante SPI y su posterior procesamiento por el microcontrolador.

4.2.3. Ensayo de la etapa de salidas digitales

Para verificar el funcionamiento de la etapa de salidas digitales, se configuró la conmutación periódica del canal 1 a una frecuencia de 1 Hz. El microcontrolador actualizó su estado mediante la interfaz SPI del VNI8200XP-32, dentro de un *superloop*, y replicó simultáneamente el estado lógico comandado sobre el pin GPIO0 de la placa.

La figura 4.2b muestra las señales registradas con el osciloscopio. El canal 1 (amarillo) corresponde a la salida industrial de 24 V, mientras que el canal 2 (celeste) representa la señal obtenida en GPIO0. La correspondencia entre ambas formas de onda permitió comprobar la transmisión del estado mediante SPI y la conmutación de la salida industrial de acuerdo con el comando establecido por el microcontrolador.



(A) Ensayo de la entrada digital.

(B) Ensayo de la salida digital.

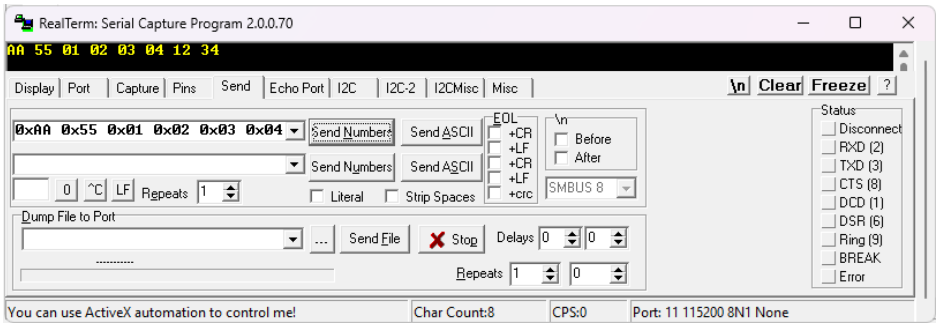
FIGURA 4.2. Señales registradas durante los ensayos de las etapas de entradas y salidas digitales.

4.2.4. Ensayo de la interfaz RS485

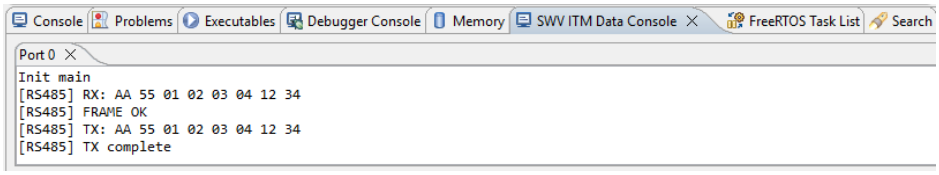
Para verificar el funcionamiento de la interfaz RS485, se conectó la placa a una computadora mediante un adaptador USB–RS485 y se configuró la comunicación a 115200 bit/s, con 8 bits de datos, sin paridad y un bit de parada. Desde RealTerm se transmitió la trama de 8 bytes `AA 55 01 02 03 04 12 34` para comprobar el intercambio de información binaria sin alteraciones.

El microcontrolador recibió la trama mediante UART7, verificó su correspondencia con la secuencia esperada y registró el resultado mediante SWO. A continuación, el transceptor RS485 se conmutó al modo de transmisión y retransmitió la misma trama hacia la computadora. Luego, retornó al modo de recepción.

La figura 4.3 muestra los resultados obtenidos. En la figura 4.3a se observa la trama retransmitida en RealTerm y, en la figura 4.3b, el registro de su recepción, validación y retransmisión mediante SWO. La coincidencia entre las tramas transmitida y recibida permitió comprobar la comunicación bidireccional y la correcta conmutación entre los modos de recepción y transmisión.



(A) Trama retransmitida visualizada mediante RealTerm.



(B) Registro de recepción, validación y retransmisión mediante SWO.

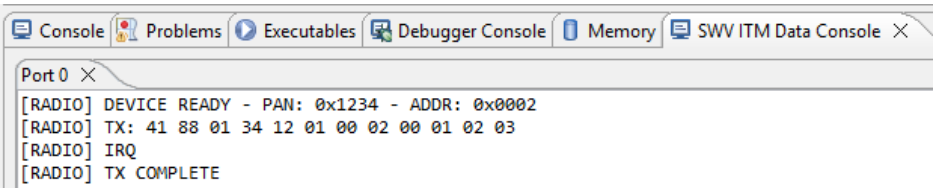
FIGURA 4.3. Resultados del ensayo de transmisión y recepción mediante la interfaz RS485.

4.2.5. Ensayo de la interfaz inalámbrica

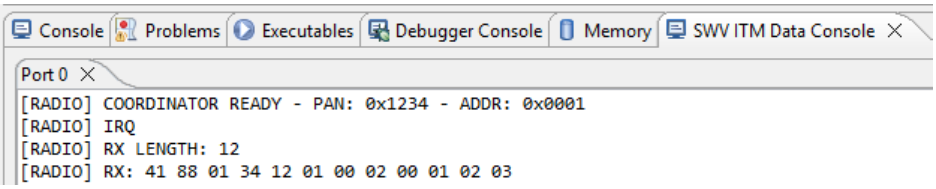
Para verificar el funcionamiento de la interfaz inalámbrica, se realizó una prueba de comunicación punto a punto entre dos placas equipadas con transceptores MRF24J40. Una de ellas se configuró como dispositivo con dirección corta `0x0002`, mientras que la segunda se configuró como coordinador con dirección `0x0001`. Ambos nodos utilizaron el identificador de red PAN `0x1234`.

Durante el ensayo, el dispositivo transmitió la trama de prueba de 12 bytes `41 88 01 34 12 01 00 02 00 01 02 03`. La transmisión se inició mediante una interrupción externa asociada al pulsador de usuario y se realizó sin solicitud de ACK. La finalización de la transmisión fue detectada mediante la interrupción generada por el MRF24J40 y registrada mediante SWO. En el coordinador, la recepción generó una interrupción del transceptor, tras la cual se leyó la FIFO y se registraron la longitud y el contenido de la trama recibida.

La figura 4.4 muestra los registros obtenidos durante el ensayo. En la figura 4.4a se observa la trama enviada por el dispositivo y la finalización de la transmisión, mientras que en la figura 4.4b se presenta la trama recibida por el coordinador. La coincidencia entre los 12 bytes transmitidos y recibidos permitió comprobar la correcta comunicación inalámbrica entre ambos nodos, así como el funcionamiento de la interfaz SPI y de las interrupciones utilizadas para gestionar el MRF24J40.



(A) Registro de la transmisión en el nodo dispositivo mediante SWO.



(B) Registro de la recepción en el nodo coordinador mediante SWO.

FIGURA 4.4. Registros obtenidos durante el ensayo de comunicación punto a punto mediante IEEE 802.15.4.

4.3. Ensayos de firmware

En esta sección se presentan los ensayos realizados para verificar el funcionamiento de los controladores de firmware y caracterizar los tiempos de ejecución de las operaciones relevantes del sistema. Las pruebas comprenden la validación unitaria de los controladores y la evaluación temporal de las operaciones de adquisición, actuación y comunicación inalámbrica.

4.3.1. Pruebas unitarias

Las pruebas unitarias se realizaron sobre los controladores de firmware correspondientes a las entradas digitales, las salidas digitales y las interfaces de comunicación RS485 e inalámbrica. Para su ejecución se utilizó Ceedling, junto con Unity como framework de pruebas y CMock para simular las funciones de acceso al hardware. Esta metodología permitió verificar el comportamiento de los controladores de forma aislada, sin depender de los dispositivos físicos.

Los ensayos contemplaron las operaciones nominales, la validación de parámetros, el tratamiento de condiciones de error y la interpretación de la información intercambiada con los periféricos. Las pruebas se organizaron en cuatro conjuntos, correspondientes a los controladores SCLT3-8BT8, VNI8200XP-32, RS485 y MRF24J40.

Controlador de entradas digitales SCLT3-8BT8

Se desarrollaron nueve pruebas unitarias sobre el archivo `sclt38bt8.c` para verificar la adquisición y el procesamiento de las tramas recibidas desde el SCLT3-8BT8. Los casos contemplaron la validación de punteros, la propagación de errores de comunicación y la decodificación de una trama válida con el patrón de entradas `0x55`.

Además, se verificó la detección de condiciones anómalas mediante tramas simuladas, entre ellas la pérdida de alimentación, los errores en los bits de parada, la paridad incorrecta, la subtensión y la sobretemperatura. Estas pruebas permitieron comprobar la interpretación de los campos de diagnóstico y la devolución de los códigos de error correspondientes.

Controlador de salidas digitales VNI8200XP-32

Se desarrollaron nueve pruebas unitarias sobre el archivo `vni8200xp32.c` para verificar la generación de comandos y el procesamiento de las respuestas del VNI8200XP-32. Se comprobó la construcción de una trama válida para actualizar las salidas con el patrón `0x55`, así como la correcta interpretación de una respuesta sin fallas.

Los casos adicionales evaluaron la propagación de errores de comunicación, la detección de paridad incorrecta y el reconocimiento de las condiciones de diagnóstico reportadas por el integrado. Estas comprendieron fallas de comunicación SPI, realimentación del convertidor DC-DC, advertencia de temperatura, señal de alimentación y sobretemperatura de canal.

Controlador de comunicación RS485

Se desarrollaron dieciocho pruebas unitarias sobre el archivo `rs485.c` para verificar el funcionamiento de la comunicación RS485, implementada mediante operaciones de transmisión y recepción por interrupciones. Los ensayos contemplaron la inicialización, la selección del modo de recepción y el rechazo de operaciones solicitadas en estados no permitidos.

Se verificaron el inicio y la finalización de las transmisiones, la recepción de datos y su disponibilidad para la aplicación. También se evaluaron las restricciones asociadas al funcionamiento semidúplex, entre ellas el rechazo de operaciones

simultáneas, la cancelación de transmisiones y recepciones activas y la recuperación del estado del controlador ante errores de UART.

Controlador de comunicación inalámbrica MRF24J40

Se desarrollaron veinticuatro pruebas unitarias sobre el archivo `mrf24j40.c` para verificar las principales operaciones del transceptor MRF24J40. Los ensayos comprendieron la inicialización del transceptor, la configuración como coordinador o dispositivo y la escritura del identificador de red PAN y de la dirección corta.

También se evaluaron el procesamiento de interrupciones, la lectura y validación de tramas desde la FIFO de recepción y la carga de tramas en la FIFO de transmisión. Se verificó la configuración de solicitudes de reconocimiento ACK y la interpretación de los resultados de transmisión, incluidas las operaciones satisfactorias, el agotamiento de reintentos y las fallas de evaluación de canal. Asimismo, se comprobó el tratamiento de parámetros inválidos y la propagación de errores de acceso al hardware.

Resultados de las pruebas unitarias

Los resultados de la ejecución de los cuatro conjuntos de pruebas se presentan en la tabla 4.3. En total, se ejecutaron 60 pruebas unitarias, todas satisfactorias, sin registrarse fallos ni casos ignorados.

TABLA 4.3. Resultados de las pruebas unitarias de los controladores de firmware.

Controlador	Pruebas	Fallos	Ignoradas
SCLT3-8BT8	9	0	0
VNI8200XP-32	9	0	0
RS485	18	0	0
MRF24J40	24	0	0
Total	**60**	**0**	**0**

Los resultados permitieron verificar el comportamiento de los controladores frente a las condiciones contempladas en los casos de prueba. Esta validación se complementó con los ensayos sobre hardware y las mediciones temporales, dado que las pruebas unitarias no incluyeron la ejecución de las funciones reales de acceso a los periféricos.

4.3.2. Evaluación temporal del firmware

Para caracterizar el desempeño temporal del sistema, se realizaron mediciones sobre las operaciones de adquisición de entradas digitales, actualización de salidas digitales y comunicación inalámbrica. Los ensayos se efectuaron sobre el hardware de desarrollo. Para las operaciones locales se utilizó el contador de ciclos DWT, mientras que la latencia de comunicación entre dispositivos se determinó mediante un osciloscopio digital.

Tiempo de procesamiento de las salidas digitales

El tiempo de procesamiento de las salidas digitales se evaluó mediante el contador de ciclos DWT del núcleo Cortex-M. La medición se realizó directamente sobre la ejecución de la función `vni8200xp32_write_outputs()`, encargada de actualizar el VNI8200XP-32. Se excluyeron el tiempo de espera en la cola de mensajes y la latencia asociada a la planificación de tareas.

Se generaron 1000 solicitudes consecutivas de actualización desde la tarea de aplicación, alternando los patrones `0x00` y `0xFF`, con una frecuencia de núcleo de 50 MHz. Todas las operaciones finalizaron correctamente, sin registrarse errores durante el ensayo.

El tiempo mínimo observado fue de 68,84 μs, el tiempo medio de 82,2 μs y el máximo de 90,12 μs. La reducida diferencia entre los valores mínimo, medio y máximo indica un comportamiento temporal estable durante la actualización de las salidas digitales.

Tiempo de adquisición de las entradas digitales

El tiempo de adquisición de las entradas digitales se evaluó mediante el contador de ciclos DWT del núcleo Cortex-M. La medición se realizó directamente sobre la ejecución de la función `sclt38bt8_read_inputs()`, encargada de efectuar la lectura del SCLT3-8BT8. Se excluyeron la actualización posterior de la imagen de entradas, el acceso al mutex y el período de espera de la tarea.

Se registraron 1000 adquisiciones consecutivas con una frecuencia de núcleo de 50 MHz. Todas las operaciones finalizaron correctamente, sin registrarse errores durante el ensayo.

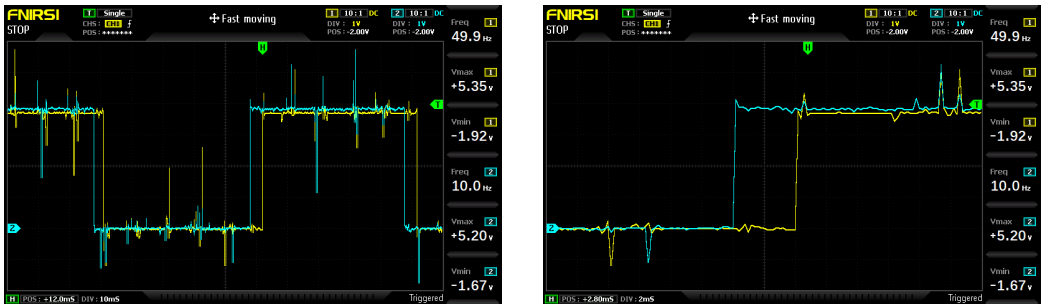
El tiempo mínimo observado fue de 59,78 μs, el tiempo medio de 60,22 μs y el máximo de 69,38 μs. La reducida diferencia entre los valores mínimo, medio y máximo indica un comportamiento temporal estable durante la adquisición de las entradas digitales.

Latencia de la comunicación inalámbrica

La latencia de la comunicación inalámbrica entre una estación remota y la estación central se evaluó mediante un osciloscopio digital. Para ello, se instrumentó el firmware de ambos dispositivos mediante una salida GPIO utilizada como referencia temporal. En la estación remota, la salida conmuta inmediatamente antes de iniciar la transmisión mediante el MRF24J40, mientras que en la estación central lo hace después de recibir y validar correctamente la trama.

El ensayo se realizó mediante transmisiones periódicas de tramas IEEE 802.15.4. La figura 4.5a muestra una vista general de las señales registradas, mientras que la figura 4.5b presenta una ampliación de los flancos utilizados para determinar la latencia. En ambas capturas, el canal 2 (celeste) corresponde a la estación remota transmisora y el canal 1 (amarillo) a la estación central receptora.

Se observaron tiempos de aproximadamente 4 ms entre los flancos correspondientes. Este intervalo comprende las operaciones de acceso al transceptor, la transmisión inalámbrica y el procesamiento de la recepción hasta la validación de la trama. El resultado permite caracterizar la latencia extremo a extremo del enlace en las condiciones del ensayo.



(A) Vista general de las señales registradas.

(B) Vista ampliada de los flancos de referencia.

FIGURA 4.5. Señales registradas mediante osciloscopio durante la medición de la latencia de comunicación inalámbrica entre la estación remota y la estación central.

4.4. Ensayos de integración

Bibliografía

[1] Martin Wollschlaeger, Thilo Sauter y Juergen Jasperneite. «The Future of Industrial Communication: Automation Networks in the Era of the Internet of Things and Industry 4.0». En: *IEEE Industrial Electronics Magazine* (2017).

[2] P. Marcon et al. «Communication Technology for Industry 4.0». En: *2017 Progress In Electromagnetics Research Symposium - Spring (PIERS)* (2017).

[3] Dapynhunlang Shylla et al. «Embedded Systems in Industrial Automation 4.0». En: *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (2023).

[4] Ian Hernández Morales. «Brief Overview of Embedded Systems for Industry 4.0 Applications and Networks». En: *2023 IEEE Integrated STEM Education Conference (ISEC)* (2023).

[5] X. Li et al. «A Review of Industrial Wireless Networks in the Context of Industry 4.0». En: *Wireless Networks* (2017).

[6] PROFIBUS and PROFINET International. *PROFIBUS*. Disponible: 2026-05-10. URL: https://www.profibus.com/technologies/profibus.

[7] PROFIBUS and PROFINET International. *PROFINET*. Disponible: 2026-05-10. URL: https://www.profibus.com/technologies/profinet.

[8] EMERSON. *Industrial wireless technology*. Disponible: 2026-05-10. URL: https://www.emerson.com/es/measurement-instrumentation/catalog/industrial-wireless-technology.

[9] PHOENIX CONTACT. *Trusted Wireless*. Disponible: 2026-05-10. URL: https://www.phoenixcontact.com/es-pc/tecnologias/tecnologias-comunicacion/trusted-wireless.

[10] SIEMENS. *SCALANCE*. Disponible: 2026-05-10. URL: https://www.siemens.com/en-us/products/scalance/.

[11] FIELDCOMM GROUP. *HART PROTOCOL SPECIFICATIONS*. Disponible: 2026-05-10. URL: https://www.fieldcommgroup.org/hart-specifications.

[12] International Society of Automation (ISA). *ISA100, Wireless Systems for Automation*. Disponible: 2026-05-10. URL: https://www.isa.org/standards-and-publications/isa-standards/isa-standards-committees/isa100.

[13] Connectivity Standards Alliance (CSA). *Zigbee The Full-Stack Solution for All Smart Devices*. Disponible: 2026-05-10. URL: https://csa-iot.org/all-solutions/zigbee/.

[14] *IEEE Standard for Low-Rate Wireless Networks*. IEEE, 2015. DOI: 10.1109/IEEESTD.2016.7460875.

[15] Lucas S. Kirschner. *Sistema modular para monitoreo y control industrial con comunicación inalámbrica y capacidades de automatización embebida*. 2025.

[16] STMicroelectronics. *STM32H562xx and STM32H563xx*. Disponible: 2026-05-17. URL: https://www.st.com/resource/en/datasheet/stm32h563ri.pdf.

[17] STMicroelectronics. *STMicroelectronics*. https://www.st.com/. (Visitado 17-05-2026).

[18] STMicroelectronics. *Introduction to the system architecture and performance in the STM32H5 MCUs*. Disponible: 2026-05-17. URL: https://www.st.com/

resource/en/application_note/an5872-introduction-to-the-system-architecture-and-performance-in-the-stm32h5-mcus-stmicroelectronics.pdf.

[19] ARM. *Arm® Cortex®-M33 Processor Technical Reference Manual*. Disponible: 2026-05-17. URL: https://developer.arm.com/documentation/100230.

[20] STMicroelectronics. *RM0481 Reference manual*. Disponible: 2026-05-17. URL: https://www.st.com/resource/en/reference_manual/rm0481-stm32h52333xx-stm32h56263xx-and-stm32h573xx-armbased-32bit-mcus-stmicroelectronics.pdf.

[21] STMicroelectronics. *Integrated Development Environment for STM32*. https://www.st.com/en/development-tools/stm32cubeide.html. (Visitado 17-05-2026).

[22] STMicroelectronics. *STM32 HAL/LL Drivers Documentatio*. https://dev.st.com/stm32cube-docs/stm32u5-hal2/2.0.0-beta.1.1/index.html. (Visitado 17-05-2026).

[23] AWS. *FreeRTOS™*. https://www.freertos.org/. (Visitado 17-05-2026).

[24] STMicroelectronics. *Protected digital input termination with serialized state transfer*. https://www.st.com/en/protections-and-emi-filters/sclt3-8bt8.html. (Visitado 17-05-2026).

[25] STMicroelectronics. *Octal high side smart power solid state relay with serial/parallel selectable interface on chip*. https://www.st.com/en/power-management/vni8200xp-32.html. (Visitado 17-05-2026).

[26] Texas Instruments. *SN65HVD82 5V-Supply RS-485 with IEC ESD Protection*. https://www.ti.com/product/SN65HVD82. (Visitado 17-05-2026).

[27] Microchip. *MRF24J40MA*. https://www.microchip.com/en-us/product/MRF24J40MA. (Visitado 17-05-2026).

[28] Microchip. *LAN8742A*. https://www.microchip.com/en-us/product/LAN8742A. (Visitado 17-05-2026).

[29] Wurth Elektronik. *1769405341*. https://www.we-online.com/components/products/datasheet/1769405341_valid_from_2026-08-31.pdf. (Visitado 27-07-2026).

[30] STMicroelectronics. *STM32 Nucleo-144 development board with STM32H563ZI MCU, supports Arduino, ST Zio and morpho connectivity*. https://www.st.com/en/evaluation-tools/nucleo-h563zi.html. (Visitado 17-05-2026).

[31] STMicroelectronics. *Industrial input/output expansion board based on VNI8200XP and CLT01-38SQ7 for STM32 Nucleo*. https://www.st.com/en/evaluation-tools/x-nucleo-plc01a1.html. (Visitado 17-05-2026).

[32] Arm. *CMSIS-RTOS2 Real-Time Operating System API*. https://arm-software.github.io/CMSIS_6/latest/RTOS2. (Visitado 08-07-2026).

[33] Internet Engineering Task Force. *Requirements for Internet Hosts – Communication Layers*. https://datatracker.ietf.org/doc/html/rfc1122. (Visitado 17-05-2026).

[34] NonGNU. *Lightweight IP stack*. https://www.nongnu.org/lwip. (Visitado 17-05-2026).

[35] TIA. *TIA – Telecommunications Industry Association*. https://tiaonline.org/. (Visitado 17-05-2026).

[36] AMD. *Reduced Media Independent Interface (RMII)*. https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/reduced_media_independent_interface.html. (Visitado 17-05-2026).

[37] AMD. *Introduction to the ARM Serial Wire Debug (SWD) protocol*. https://developer.arm.com/documentation/ihi0031/a/The-Serial-Wire-Debug-

Port--SW-DP-/Introduction-to-the-ARM-Serial-Wire-Debug--SWD--protocol. (Visitado 17-05-2026).

[38] IEEE. *IEEE Standard for Test Access Port and Boundary-Scan Architecture*. https://sagroups.ieee.org/1149-1/. (Visitado 17-05-2026).

[39] *Industrial-process Measurement and Control – Programmable Controllers – Part 2: Equipment Requirements and Tests*. International Electrotechnical Commission, 2017.

[40] *Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems*. Telecommunications Industry Association, 1998.

[41] *IEEE Standard for Ethernet*. IEEE, 2022.

[42] *Generic Standard on Printed Board Design*. IPC, 2003.

[43] *Sectional Design Standard for Rigid Organic Printed Boards*. IPC, 1998.

[44] *Generic Requirements for Surface Mount Design and Land Pattern Standard*. IPC, 2005.

[45] *Graphic Symbols for Electrical and Electronics Diagrams (Including Reference Designation Letters)*. IEEE, 1975.